

2026 상반기 DFIR 기초 역량 강화 세미나

기기(이미징)

2026-02-24

김은채, 조민혁



고려대학교 정보보호대학원
Korea University
School of Cybersecurity



DFRC Korea University
Digital Forensic Research Center

- ❑ 저장장치
- ❑ 저장장치 인터페이스
- ❑ 이미징
- ❑ PC 데이터 수집
- ❑ 컴퓨터 분해·조립 및 기본 설정

Keywords

ROM / RAM

HDD / SSD

Protocol

Bus

Form Factor

물리 이미지

논리 이미지

저장장치

개요

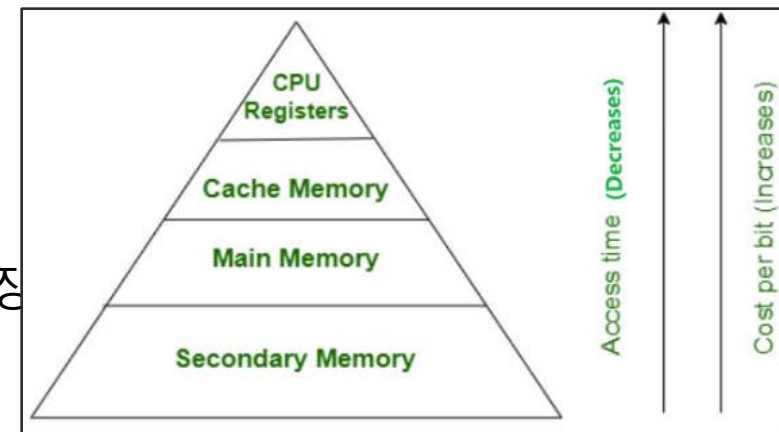
■ 저장장치 정의

- 디지털 기기에서 생성된 디지털 데이터를 영구적 / 반영구적 형태로 읽기, 쓰기, 보관할 수 있는 하드웨어
- 주기억장치 (Main Memory)
 - 프로그램 실행에 필요한 데이터, 명령어를 저장하고, CPU가 직접 접근할 수 있는 기억장치
 - 빠른 처리속도, 낮은 용량
- 보조기억장치 (Secondary Memory)
 - 데이터와 프로그램을 장기간 저장하기 위한 기억장치
 - 주기억장치에 비해 느린 처리속도, 큰 용량

개요

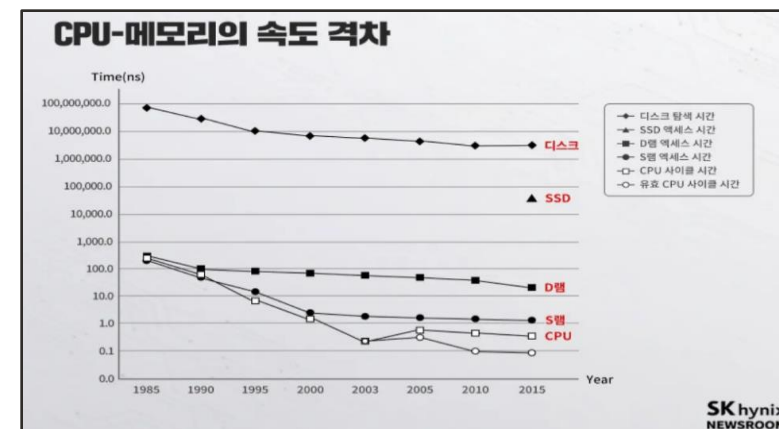
■ 메모리 계층 구조[추가]

- 속도, 가격, 용량에 따른 계층적으로 나눈 구조
 - 레지스터: CPU 내부에서 연산에 필요한 데이터 저장
 - 캐시: 주기억장치 간 병목 해결을 위해 자주 접근되는 데이터 저장
 - 주기억장치: 실행 중인 프로그램 및 데이터 저장
 - 보조기억장치: 영구적 / 반영구적 데이터 저장



■ 계층구조의 탄생

- CPU와 저장장치 간의 속도 차이 문제를 해결하기 위한 구조적 해결책



개요

■ 주기억장치 vs 보조기억장치: 가격 차이 원인[추가]

■ 집적도 (Density)

- 주기억장치: 1 cell = 1 bit → 비트 가격 고가
- 보조기억장치: 1 cell = SLC / MLC / TLC / QLC, 여러 bit 저장 가능
→ 비트 가격 저가

■ 수율 (Yield)

- 주기억장치: 타이밍 오차, 안정성, 결함 → 수율 기준 엄격
- 보조기억장치: ECC 사용, 일부 bit 오류 허용 가능 → 수율 기준 상대적으로 유연
 - ECC (Error Correcting Code): 쓰기 시 ECC 알고리즘으로 계산된 값을 기록, 읽기 시 기록된 값으로 오류 검사 및 복구

RAM

■ 정의

- Random Access Memory
- CPU가 현재 사용 중인 데이터를 저장하는 휘발성 메모리의 한 형태
 - Random: CPU가 원하는 메모리 주소로 이동 가능
 - 휘발성: 전원이 꺼지는 순간 모든 데이터 사라짐
- CPU가 캐시 접근 후 캐시에 없으면 RAM 접근
- SSD보다 수천 배 빠르지만, 크기가 작고 비쌈



RAM

■ RAM 종류

■ SRAM (Static RAM)

- 기술: 비트 당 6개의 트랜지스터로 구성된 플립플롭 (Flip-Flop) 회로 사용
- 역할: CPU 캐시 (L1, L2, L3)에 사용
- 장점: 매우 빠름, Refresh 불필요
- 단점: DRAM 보다 비쌈, DRAM 보다 저용량

■ DRAM (Dynamic RAM)

- 기술: 1 트랜지스터 + 1 커패시터 구조 사용
- 역할: 메인 시스템 메모리로 사용
- 장점: SRAM 보다 저렴함, SRAM 보다 고용량
- 단점: SRAM 보다 느림, 커패시터의 전하가 누설되므로 Refresh 필요

RAM

■ RAM 종류

■ SDRAM (Synchronous DRAM)

- 발전 배경: SDRAM 이전의 RAM은 CPU 클럭과 독립적으로 동작
- 역할: CPU의 시스템 클럭과 동기화되어 동작
- 영향: 메모리 컨트롤러가 데이터 준비 시점을 예측할 수 있어 성능 향상

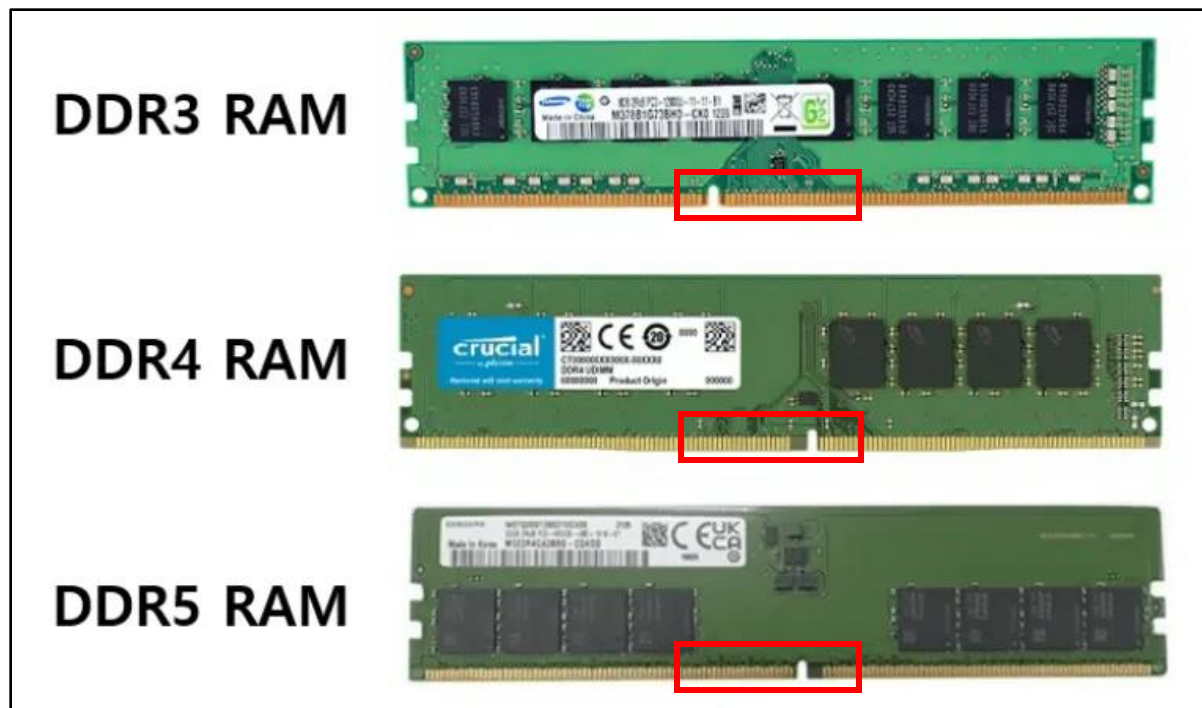
■ DDR (DDR3 ~ DDR5)

- DDR (Double Data Rate)은 클럭 신호의 상승 에지와 하강 에지 모두에서 데이터를 전송하는 SDRAM
- 버전이 증가할 수록 대역폭과 전송 속도는 증가, 전압은 감소

버전	전압	프리패치	대역폭 (GB/s)	전송 속도 (MT/s)
DDR3	1.5V	8n	6.4 ~ 17.0	800 ~ 2133
DDR4	1.2V	8n	12.8 ~ 25.6	1600 ~ 3200
DDR5	1.1V	16n	25.6 ~ 51.2	3200 ~ 6400

RAM

- **DDR 선택 시 고려사항**
 - DDR은 메인보드에 장착해서 사용
 - 세대 별로 장착 부분 (노치) 위치가 상이
 - 메인보드에서 물리적으로 혼용을 차단
 - 메인보드가 지원하는 세대를 파악 후 DDR을 장착해야 호환성 확보 가능



RAM

■ 뱅크 (Bank)[추가]

- 메모리 칩 내부를 논리적으로 나눈 하위 단위
 - 독립적으로 제어할 수 있는 가장 작은 단위
 - 각 뱅크는 Row Buffer를 보유
- CPU가 데이터를 요청 시 뱅크의 동작 과정
 - Activate: Bank의 Row 열기
 - Read / Write: 특정 Column 선택 후 전송
 - Precharge: Bank의 Row 닫기
- 연속된 메모리 요청 시 뱅크의 동작 과정
 - Row-Buffer Hit: 이전에 열려 있던 행 버퍼 이용, 빠름
 - Row-Buffer Miss: 이전에 열려 있던 행 버퍼 닫은 후 새로운 행 버퍼 여는 과정 필요, 느림

RAM

■ DDR 뱅크 충돌[추가]

■ 성능 영향

- Precharge → Activate 과정
 - 지연 시간 증가

■ 완화 기법

- 병렬성 증가 → 뱅크 수를 늘리고, Bank Group으로 구성하면
 - 여러 경로로 동시에 데이터 접근이 가능
 - 병렬성 (Parallelism)이 향상
- DDR4: 16개 Bank (4개 Bank Group)
- DDR5: 32개 Bank (8개 Bank Group)

RAM

■ DDR의 Prefetch 기능[추가]

■ 개념 및 동작 방식

- 처리량 (Throughput)을 극대화하기 위한 RAM 칩 내부의 저수준 하드웨어 구조
- DDR3, DDR4, DDR5 칩 내부에 존재
- 동작 방식: n 단위의 아키텍처
- 1비트가 아닌 데이터 블록을 ($8n / 16n$) 한 번에 가져옴
- 가져온 넓은 데이터 블록을 버퍼에 저장
- 고속 I/O 라인을 통해 외부로 빠르게 전송
- DDR3 / DDR4 ($8n$): 하나의 64바이트 CPU 캐시 라인을 효율적으로 채우도록 최적화
- DDR5 ($16n$): 고성능 프로세서를 지원하기 위해 16단위로 확장

RAM

■ L1 / L2 / L3 Cache[추가]

■ 정의

- 빠른 CPU와 느린 RAM 사이의 속도 격차를 줄여주는 고속 온보드 메모리 계층
- 속도, 용량, 비용의 균형을 맞추기 위해 분리
- CPU에 가까울수록 빠름 → 크기는 작아짐

캐시 레벨	위치	접근 속도	일반적인 크기	접근 방식	설명
L1	코어 내부	가장 빠름	~ 64KB	코어 전용	CPU가 현재 즉시 처리 중인 명령어와 데이터 저장
L2	코어 내부	매우 빠름	~1MB	코어 전용	CPU가 가까운 미래에 필요로 할 가능성이 있는 데이터 저장
L3	코어 외부	빠름	~ 32MB	공유 (모든 코어)	하나의 CPU 칩 내 모든 코어가 공유 → 코어 간 데이터 교환 매개체로 사용

RAM

- 단위 MT/s[추가]
 - Mega transfer per second
 - 유효 메모리 속도
 - 데이터 전송률을 의미하며, 구체적으로 초당 실제로 데이터가 몇 번 전송되는지를 의미
 - MHz과 다름
 - 클럭 속도를 의미하며, 구체적으로 초당 전기 신호가 몇 번 펄스를 발생시키는지 의미
 - DDR에서는 1 클럭 사이클 (MHz)당 2번의 데이터 전송 (MT/s)이 이루어짐
 - $MT/s = MHz * 2$

ROM

■ 정의

- Read Only Memory
- 한번 기록한 데이터가 전원 유지와 관계없이 비휘발성으로 기록되는 메모리
- 기록된 데이터를 읽기만 가능하며, 삭제나 수정이 불가
- 주로 BIOS, 펌웨어를 저장하는데 사용
 - BIOS (Basic Input/Output System): 소프트웨어와 하드웨어 간의 설정 및 정보전달의 매개 역할 수행
- Mask ROM, PROM, EPROM, EEPROM, Flash Memory 존재

ROM

■ ROM 종류

■ PROM (Programmable ROM)[추가]

- 생산 후 사용자가 한 번만 기록 가능한 ROM
- 소량 생산 가능, 개발 및 테스트에 유연한 작용

■ EPROM (Erasable PROM)

- 자외선 (UV)를 사용하여 기록된 데이터를 지우고 다시 기록 가능한 ROM
- 구조상 부분 삭제 불가
- 기판에서 EPROM을 제거한 후 삭제

ROM

■ ROM 종류

■ EEPROM (Electrically EPROM)

- 전기적으로 지우고 기록 가능한 ROM
- 바이트 단위 부분 삭제 지원
- 회로에 쪼흔 채로 삭제 가능

■ PROM의 발전 원인[추가]

- ROM은 대량생산으로 낮은 원가에 공급 가능
- 생산된 ROM에 오류, 업데이트 등의 문제 발생시 전량 폐기의 위험성
- 소량 생산 시 비용 상승 문제등으로 수정 가능한 ROM에 대한 시장 요구

ROM

■ Flash Memory

- 전기적으로 삭제 및 기록 가능한 ROM
- 블록 단위 부분 삭제 지원
- 높은 집적도
- 회로에 쪼갬 채 삭제 가능

기능	EPROM	EEPROM	Flash Memory
휘발성	비휘발성	비휘발성	비휘발성
삭제 방식	자외선	전기	전기
삭제 범위	칩 전체	바이트 단위	블록 단위
부분 삭제	불가능	바이트 단위 가능	블록 단위 가능
삭제 시 회로 분리 여부	필요	불필요	불필요

ROM

■ RAM vs ROM 비교

- RAM은 자유로운 읽기 / 쓰기 가능, ROM은 쓰기에서 제한 존재
- RAM은 데이터를 휘발성 저장, ROM은 비휘발성 저장
- RAM은 ROM보다 빠른 속도지만 적은 용량
- RAM, ROM 특성상 사용처 상이

특징	RAM	ROM
목적	현재 실행 중인 프로그램 처리	영구적 / 반영구적인 데이터 저장
사용처	주기억장치, 캐시메모리 등	BIOS, 펌웨어 등
쓰기 여부	가능	제한적으로 가능
휘발성	휘발성	비휘발성
속도	빠름	느림
용량	저용량	대용량
종류	DDR, DRAM, SRAM, SDRAM 등	PROM, EPROM, EEPROM, Flash Memory 등

HDD

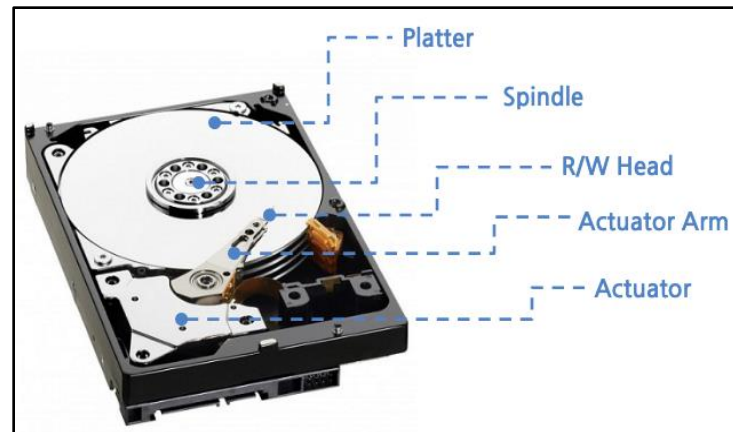
■ 정의

- Hard Disk Drive, 회전하는 디스크에 자기로 데이터를 읽고 쓰는 보조기억장치

■ 구조

- 물리적 구조

- 플래터: 원형의 금속 디스크로 자기 물질로 코팅되어 있으며 실제 데이터가 기록
- 스피들: 플래터를 회전시키는 축
- 헤드: 플래터 표면 위를 이동하는 장치로 데이터를 읽고 쓰는 역할
- 액추에이터 암: 트랙 위치로 헤드를 이동시키는 역할 수행

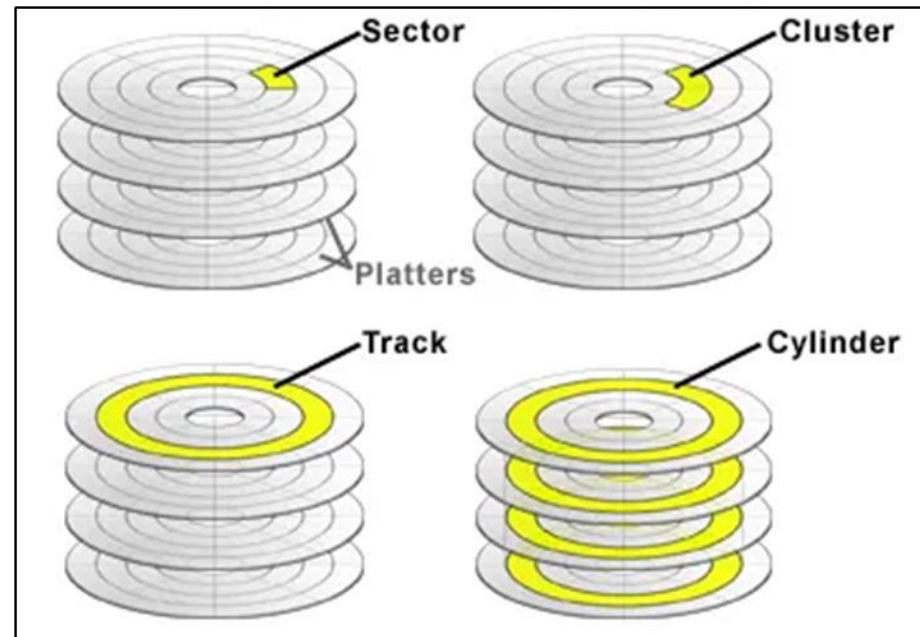


HDD

■ 정의

■ 구조

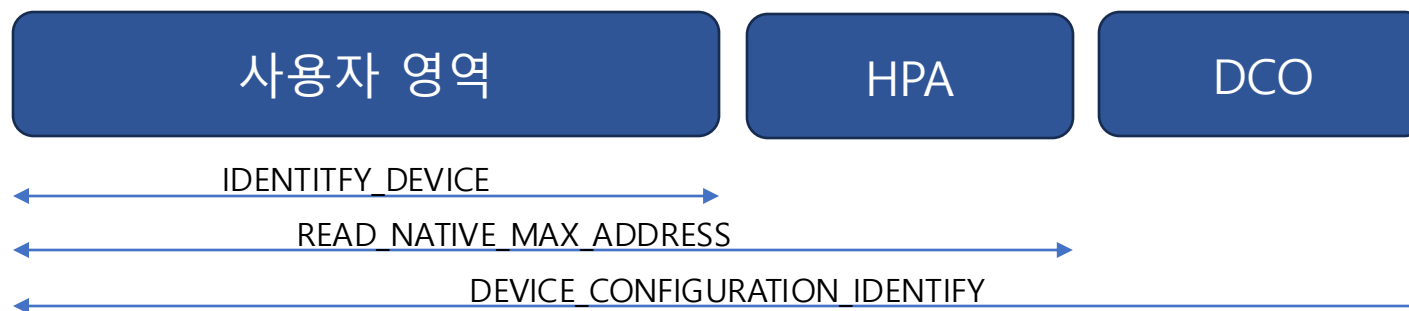
- 논리적 구조
 - 트랙: 플래터를 동심원 형태로 나눈 단위
 - 섹터: 트랙을 잘게 나눈 최소 저장 단위
 - 클러스터: 연속된 섹터를 묶어 만든 저장 단위
 - 실린더: 여러 플래터에서 같은 위치에 있는 트랙의 집합
- HDD는 외부 충격이나 자기장에 데이터가 손상되기 쉬움



HDD

■ HPA, DCO, SA

- HPA (Host Protected Area): OS 수준에서 숨겨져 있으며 시스템 부팅, 복구를 위한 숨겨진 영역
- DCO (Device Configuration Overlay): 하드웨어 수준에서 디스크 용량을 제한해서 숨겨진 영역
 - IDENTIFY_DEVICE: 현재 BIOS를 통해 접근 가능한 영역 리턴
 - READ_NATIVE_MAX_ADDRESS: 실제 디스크 전체 영역을 리턴, HPA 영역 확인 가능
 - DEVICE_CONFIGURATION_IDENTIFY: DCO 영역 확인 가능
- SA (Service Area): HDD가 스스로를 관리하기 위한 펌웨어가 저장되므로 사용자 접근 불가



HDD

■ HPA, DCO, SA

■ HPA (Host Protected Area)

- OS 수준에서 숨겨져 있으며 시스템 부팅, 복구를 위한 숨겨진 영역
 - 물리적으로는 존재, 논리적으로 사용자에게 숨김
- 시스템 부팅, 복구, 진단 도구를 위한 공간이며 루트킷(rootkit)으로 악용 여지 존재
- HPA와 관련된 ATA 명령어
 - IDENTIFY DEVICE: 현재 BIOS를 통해 접근 가능한 영역 리턴
 - SET MAX ADDRESS: BIOS를 통해 접근 가능한 영역 설정
 - READ NATIVE MAX ADDRESS: 실제 디스크 전체 영역 리턴

HDD

■ HPA, DCO, SA

■ DCO (Device Configuration Overlay)

- HDD의 하드웨어 기능과 최대 용량을 제한하는 하드웨어 설정 영역
- HPA보다 더 낮은 레벨에서 용량을 제한
- BIOS를 통해 확인되지 않고 ATA 명령을 통해 접근 필요
- DCO와 관련된 ATA 명령어
 - DEVICE CONFIGURATION SET: DCO 값 설정
 - DEVICE CONFIGURATION IDENTITY: DCO 설정 상태 확인
 - DEVICE CONFIGURATION RESTORE: DCO 제조사 기본값 복원

HDD

■ HPA, DCO, SA

■ SA (Service Area, System Area)

- 플래터의 특정 트랙에 존재하는 디스크 전용 시스템 영역
- 주요 구성 요소
 - 펌웨어 코드: 플래터 회전, 읽기/쓰기 명령 등 HDD 작동을 제어하는 핵심 코드
 - 결함 목록: 플래터에서 발견된 결함 기록
 - S.M.A.R.T. 데이터: 전원 켜짐 시간, 오류율, 온도 등의 지표 추적 등 자가 점검 데이터 기록
 - 존 테이블: 플래터 전체에 걸쳐 구역별 기록 규격을 기록하고, 헤드가 각 영역에 맞도록 보정
 - 드라이브 별 적응형 데이터: 내부 구성 요소에 대한 고유한 보정 데이터로, 헤드가 데이터 트랙을 안정적으로 읽도록 보장

HDD

■ LBA 관점에서의 HPA, DCO, SA[추가]

■ HPA

- 물리적으로 HPA 영역은 존재하지만, 논리적으로 사용자 입장에서는 확인 불가
- 디스크가 보고하는 Current Max LBA 값에 따라 OS가 인식하는 디스크 용량 결정

■ DCO

- DCO는 디스크가 보고하는 Native Max LBA를 설정
- 장치의 최대 용량을 하드웨어 수준에서 제한

■ SA

- LBA 주소 변환 경로에 SA 영역은 미포함
- LBA 관점에서 완전 접근 불가, 전문 장비를 통해서만 접근 가능

HDD

■ HPA의 특징[추가]

- 논리적으로 사용자가 볼 수 있는 영역에 포함되지 않음
 - HPA 공간 인식 부재
- OS 포맷 시 HPA 영역은 OS 관리 밖의 영역 가능
 - 새로운 OS 설치 시에도 HPA 영역에 루트킷 존재 가능

■ 루트킷의 HPA 침투 방식[추가]

- 취약점을 이용해 시스템 침투 및 권한 상승
- 관리자 권한을 통해 ATA 표준 명령 사용
 - IDENTIFY_DEVICE → SET_MAX_ADDRESS → 데이터 기록 → SET_MAX_FREEZE LOCK

HDD

■ 섹터 크기 512 Bytes 크기 이유[추가]

■ 과거 CPU 연산 능력의 한계

- Bytes를 2의 거듭제곱 표현 시 비트 시프트 연산 용이 및 하드웨어 구현 단순화

■ 디스크 컨트롤러 내부 버퍼 RAM의 용량 한계

- 당시 RAM의 크기는 1KB~8KB 수준
- 섹터 크기가 매우 커질 시 RAM에 적재하기 부담

■ 크기 변화에 따른 장단점

- 크기가 너무 작을 시 용량 낭비가 적지만 헤드에서 처리되는 섹터 범위가 많아져 오버헤드 증가
- 크기가 너무 클 시 한 번에 처리할 수 있는 범위가 커져 오버헤드 감소하지만 용량 낭비 증가

HDD

■ 섹터의 물리적 단위 vs 논리적 단위[추가]

- 디스크의 실제 읽기 / 쓰기 최소 단위
 - 대부분 4KB 단위로 이루어지며, ECC 오류 정정도 함께 수행
- 디스크 인터페이스가 요청하는 입출력 최소 단위
 - 대부분의 OS / 파일시스템 / 응용프로그램과의 호환성을 유지하기 위해 512 Bytes 단위로 입출력을 수행
- 논리적 섹터 8개 → 물리적 섹터 1개로 이루어짐
- 512e 에뮬레이터가 논리적 섹터와 물리적 섹터 사이의 인터페이스 역할 수행
 - 512e: 논리적 섹터 번호를 통해 물리적 섹터 번호 및 섹터 내 오프셋을 계산

HDD

■ Bad Sector[추가]

■ 개념

- 데이터를 읽거나 쓰는 최소 단위가 정상 동작하지 않는 상태
- 읽기 / 쓰기 과정에서 오류가 발생해 ECC가 복구 시도 후 복구 실패 시 Bad Sector로 판별한 섹터

■ 종류

- 제조 과정에서 생긴 결함 → P-list에 등록
 - P-list (Primary Defect List): 제조 검사 과정에서 발견된 결함 목록, 출고 전 매핑되지 않도록 처리
- 사용 중 생긴 결함 → G-list에 등록
 - G-list (Grown Defect List): 사용 중 새로 생긴 결함 목록, Bad Sector는 해당 리스트에 추가되고 리맵됨

■ 디스크의 Bad Sector 처리 과정

- 리맵 메커니즘을 통해 논리 주소는 유지한 채 Spare 섹터를 할당 받아 사용

HDD

■ HDD의 기록 방식[추가]

■ HDD 플래터 표면

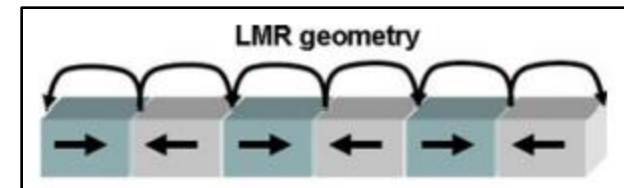
- 자성 입자들로 이루어진 얇은 자성층
- 자성 입자들의 자화 방향이 비트의 1 또는 0을 결정
 - 자화 방향: 방향과 크기를 가진 자성 입자들의 평균 방향
- HDD에서 자화 방향을 이용해 정보를 기록
- 저장 밀도를 높이기 위해 비트 셀을 작게 함
 - 너무 작아질 시, 열 때문에 자화가 의도하지 않게 뒤집혀지는 문제 발생
- 문제를 해결하기 위한 4가지 기록 방식 존재 → LMR, PMR, SMR, HAMR

HDD

■ LMR (Longitudinal Magnetic Recording)[추가]

■ 개념

- 플래터 표면과 수평 방향으로 자성 입자를 기록하는 방식
- 장점: 구조가 단순
- 단점: 탈자장에 의해 자화를 유지하기 어려워져 고밀도화 하기에는 한계가 있음
 - 탈자장: 자성체 외부의 자화에 의해 생성되는 자기장



■ 쓰기 과정

- 기록 헤드가 가한 자기장을 순간적으로 발생 → 해당 위치의 자화 방향을 한쪽으로 정렬

■ 읽기 과정

- 읽기 헤드가 플래터를 지나가며 자속 변화 감지 → 자화 변화 경계에서 큰 신호 발생
→ 해당 신호를 복호화하여 비트로 변환

HDD

■ PMR (Perpendicular Magnetic Recording)[추가]

■ 개념

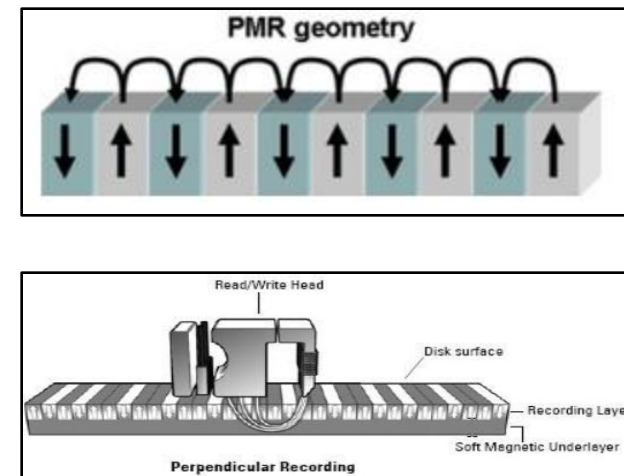
- 플래터 표면과 수직 방향으로 자성 입자를 기록하는 방식
- 장점: LMR보다 고밀도화에 유리
- 단점: 비트를 작게 할 수록 열적 안정성 문제 발생

■ 쓰기 과정

- 기록 헤드가 강한 자기장을 순간적으로 발생 → SUL이 자성 입자들을 끌어당겨 위 / 아래 중 한 방향으로 정렬
 - SUL (Soft Under Layer): 수직 방향 기록 시 자성 입자의 흩어짐 문제를 바닥 부분에서 끌어당기는 역할

■ 읽기 과정

- LMR과 동일



HDD

■ SMR (Shingled Magnetic Recording)[추가]

■ 개념

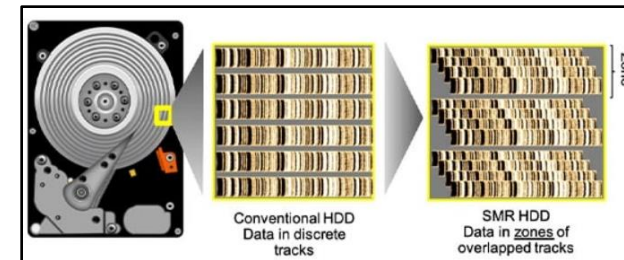
- “읽기 헤드는 좁게 읽을 수 있고, 쓰기 헤드는 크게 쓸 수 있다”라는 생각에서 등장
- 기와처럼 트랙 내에 비트를 겹쳐서 기록하는 방식
- 장점: 저장 용량 증가, 순차 쓰기에 유리
- 단점: 초기에는 성능 우수하지만, 추후 성능 감소 발생

■ 쓰기 과정

- 각 트랙 별로 기록하면서 다음 트랙 작성 시 이전 트랙과 살짝 덮어 기록
- 겹친 부분 수정 발생 시 겹친 부분 전체 메모리에 적재 후 수정하고자 하는 부분 수정 후 다시 기록

■ 읽기 과정

- 읽기 헤드는 좁게 읽을 수 있기에 이전과 읽기 과정 동일



HDD

■ HAMR (Heat-Assisted Magnetic Recording)[추가]

■ 개념

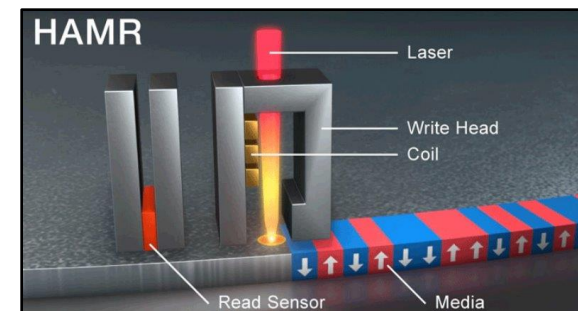
- 비트를 너무 작게 했을 때 발생하는 열적 안정성 문제 발생
- 열적 안정성에 강한 자석 기반은 쓰기 헤드로 기록 어려움
- 이에 따라 강한 자석을 사용하면서 열의 도움을 받아 작성하는 방식
- 장점: 저장 밀도 증가, 열적 안정성 문제 해결
- 단점: 헤드 내 레이저 도입에 따른 구조 복잡, 쓰기 속도 저하

■ 쓰기 과정

- 아주 작은 레이저로 비트 한 점을 순간적으로 가열하여 기록

■ 읽기 과정

- 기존과 동일



SSD

■ 개념

- NAND Flash 기반의 비휘발성 저장장치
- 기계적 이동이 없어 지연 시간이 짧아 빠른 데이터 액세스 속도와 높은 내구성 제공
- 주로 사용되는 SSD
 - 인터페이스 / 프로토콜 : SATA SSD (SATA / AHCI), NVMe SSD (PCIe / NVMe or AHCI), SAS SSD (SAS / SCSI)
 - 폼팩터: 2.5" SSD, M.2 SSD



SSD

■ 개념

■ NOR Flash vs NAND Flash

• NOR Flash

◦ 각 셀이 병렬로 Bit Line에 직접 연결되어 개별 접근 가능

→ 랜덤 읽기에 유리

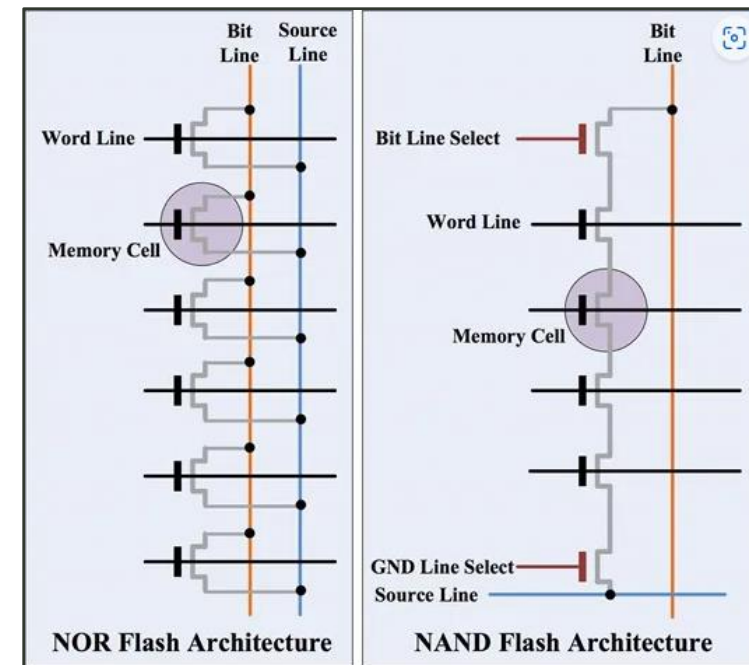
◦ Bit Lane 접점 / 배선이 늘어 고집적에 불리

• NAND Flash

◦ 여러 셀이 BL과 SL 사이에 직렬로 연결 되어 개별 접근 제한

→ 페이지 / 블록 단위 쓰기·삭제에 유리

◦ BL 접점 / 배선이 줄어 고집적 / 저가 / 대용량에 유리



구분	NOR Flash	NAND Flash
셀 연결방식	병렬	직렬
밀도	저밀도	고밀도
읽기속도	빠름	느림
쓰기/삭제 속도	느림	빠름
주 사용처	펌웨어	SSD, USB

SSD

■ 구조

■ Controller

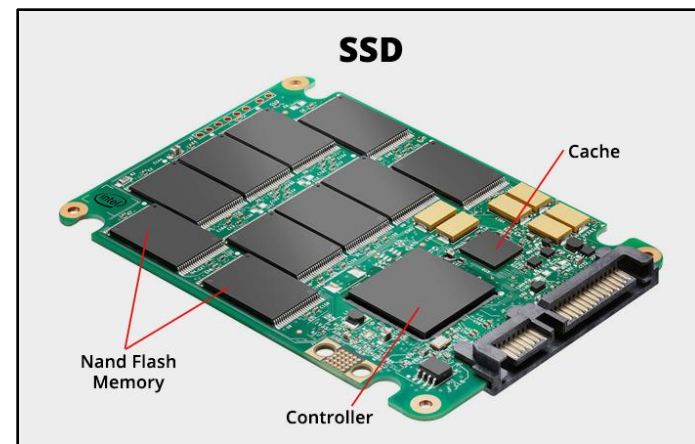
- SSD 동작을 총괄하는 핵심 제어 장치로 성능과 수명에 직접 영향
- FTL로 LBA ↔ 물리주소 매핑, 스케줄링 수행
 - 스케줄링: 들어온 I/O와 내부 작업의 우선순위를 조정해 성능 / 지연을 최적화 하는 기능

■ DRAM 캐시

- 임시 저장 영역으로 데이터 접근 속도와 성능 향상
- 주소 매핑 테이블과 자주 쓰는 데이터를 캐싱

■ NAND Flash Memory

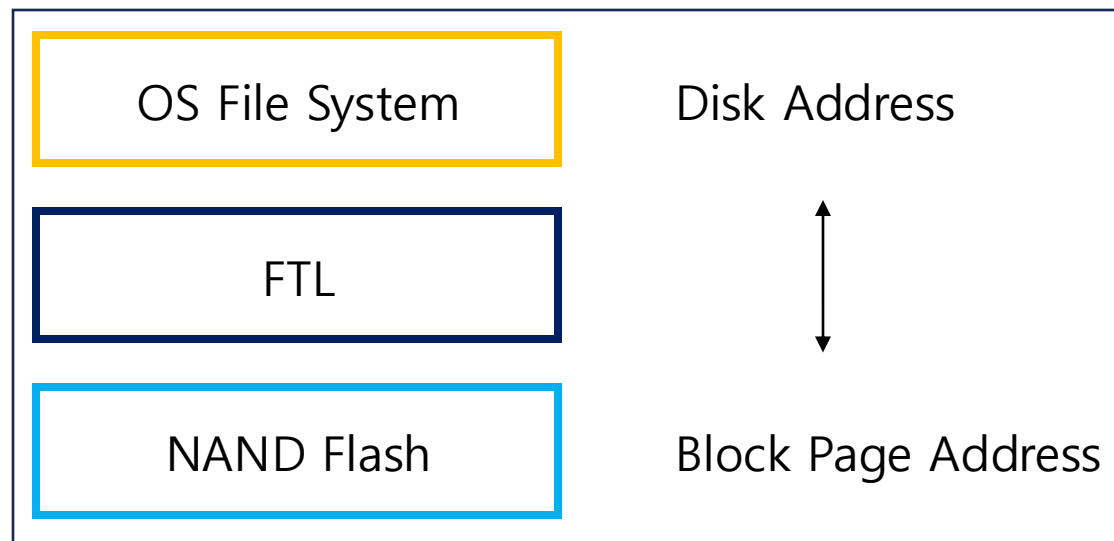
- 실제 데이터가 저장되는 비휘발성 메모리
- 플래시 특성상 블록 단위 삭제, 페이지 단위 쓰기, P/E 사이클 수명 제한 존재



SSD

■ FTL (Flash Translation Layer) 기능

- OS 관점에서 SSD가 HDD처럼 동작할 수 있도록 만들어주는 계층
- SSD 컨트롤러 내에서 동작하며, Mapping Table을 참조하여 논리적 주소를 물리적 주소로 변환
- Wear Leveling, Garbage Collection 등을 통해 SSD 성능 및 수명 최적화



SSD

■ FTL (Flash Translation Layer) 기능

■ Logical Block Mapping

- SSD는 Block과 페이지를 기준으로 동작하지만 OS는 HDD의 섹터 기반 주소 사용
- OS의 논리적 섹터 주소를 SSD의 물리적 Block과 페이지 주소로 변환

■ Wear Leveling

- Flash 메모리 셀은 제한된 P (Program) / E (Erase) 사이클 존재
- 블록 단위로 지우기 때문에 erase count를 블록단위로 모니터링하여 erase count가 낮은 블록을 우선 사용
- 데이터 기록을 전체 셀에 고르게 분산하여 수명과 안정성 향상

SSD

■ FTL (Flash Translation Layer) 기능

■ Garbage Collection

- NAND는 덮어쓰기가 불가하여 블록 내 무효 페이지가 누적
- Controller가 블록 내 유효한 페이지와 유효하지 않은 페이지를 식별[추가]
 - 페이지 당 존재하는 valid bitmap (유효하면 1, 유효하지 않으면 0)을 읽어 유효성 판별
 - 유효한 페이지들 대상으로 Mapping Table을 확인하여 복사 여부를 최종적으로 검증
- 유효 데이터만 다른 블록으로 복사 후, 원래 블록을 블록 단위로 erase하여 빈 공간 회수
- 여유 블록을 확보해 지속 쓰기 성능과 쓰기 지연을 안정적으로 유지

SSD

■ TRIM vs Garbage Collection

■ TRIM

- 무효 페이지의 논리 주소 정보를 운영체제가 SSD 컨트롤러에게 전달하는 기능
- 무효 데이터를 표시함으로써 GC의 복사량 감소

■ Garbage Collection

- SSD가 내부적으로 데이터를 정리하는 작업
- TRIM이 없어도 동작하지만 TRIM이 있을 시 불필요한 복사를 줄여 더 효율적으로 동작

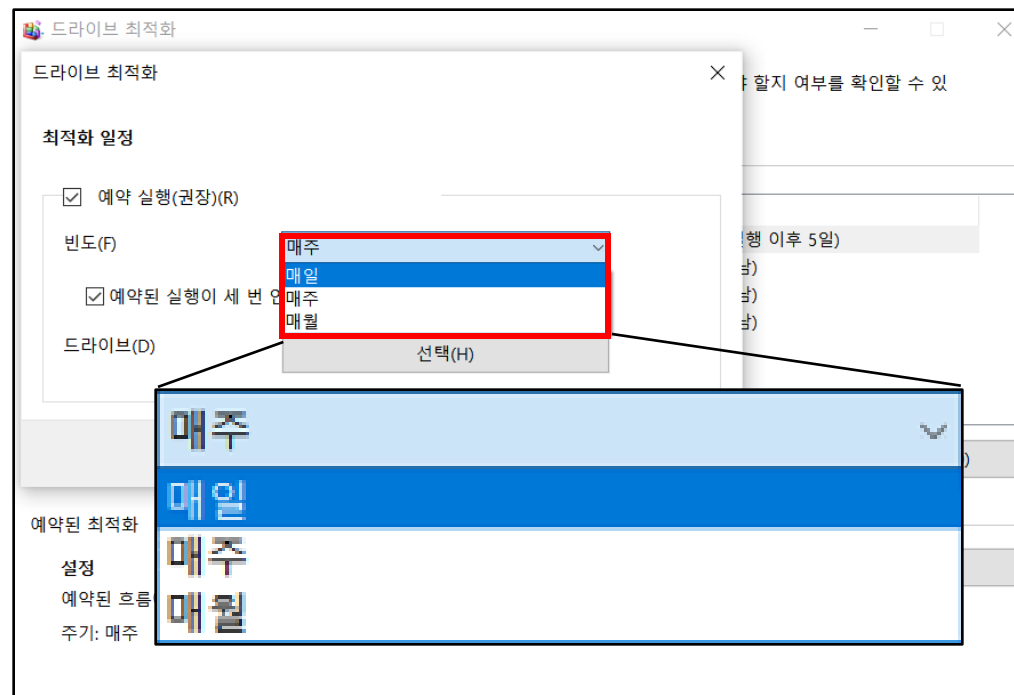
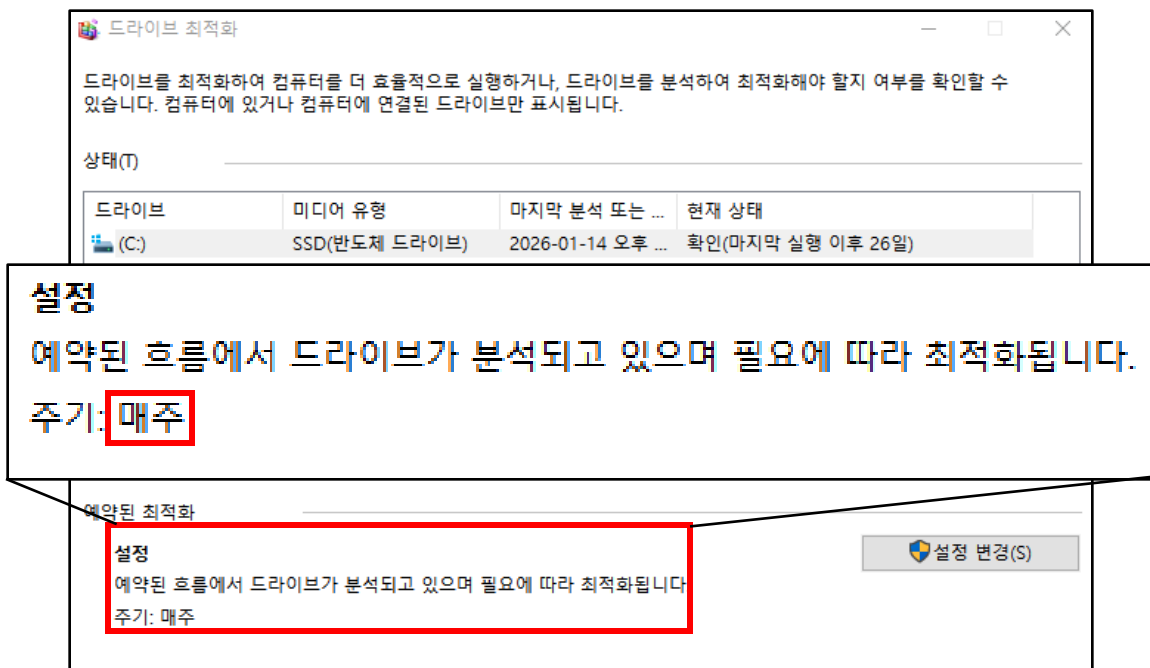
구분	TRIM	Garbage Collection
주체	운영체제	SSD 컨트롤러
역할	삭제된 데이터 정보 전달	내부적으로 데이터 정리
실행 시점	LBA에 논리적으로 free 발생 시	SSD가 자체적으로 실행
한 가지만 활성화 시	삭제된 데이터 잔존 가능	유효 데이터 삭제 가능
관계	GC 효율 향상	TRIM 있을 시 효율적으로 동작

SSD

■ [실습] TRIM

■ 활성화 주기 확인 및 변경[추가]

- 드라이브 최적화 및 조각 모음에서 활성화 주기 확인 및 변경 가능



SSD

■ [실습] TRIM

■ 활성화 및 비활성화

- 명령어: fsutil behavior set DisableDeleteNotify 0
 - fsutil: Windows의 파일 시스템 관련 고급 설정 / 정보를 다루는 명령줄 도구
 - behavior: 파일 시스템의 동작 정책 / 기본 동작을 다루는 하위 명령
 - set: 해당 동작 정책 값을 변경
 - DeleteNotify: TRIM 명령 전송
- 활성화
 - fsutil behavior set DisableDeleteNotify 0
- 비활성화
 - fsutil behavior set DisableDeleteNotify 1

```
C:\Users\eunchaeKIM>fsutil behavior query DisableDeleteNotify
NTFS DisableDeleteNotify = 0 (TRIM 작업을 저장 장치로 보낼 수 있도록 허용)
ReFS DisableDeleteNotify = 0 (TRIM 작업을 저장 장치로 보낼 수 있도록 허용)

C:\Users\eunchaeKIM>fsutil behavior set DisableDeleteNotify 1
NTFS DisableDeleteNotify = 1 (TRIM 작업을 저장 장치로 보낼 수 있도록 허용 안 함)

이 작업은 즉시 적용됩니다(다시 부팅할 필요 없음).

C:\Users\eunchaeKIM>fsutil behavior set DisableDeleteNotify 0
NTFS DisableDeleteNotify = 0 (TRIM 작업을 저장 장치로 보낼 수 있도록 허용)

이 작업은 즉시 적용됩니다(다시 부팅할 필요 없음).
```

SSD

- [실습] TRIM
 - 파일 생성
 - 파일 생성 후 FTK Imager에서 파일 관련 정보 확인
 - Start Sector: 583,049,968
 - Offset: $583,049,968 * 512 = 0x458145E000$
 - 파일 영구삭제 후 데이터 및 디스크 용량 확인
 - TRIM 이후 데이터 및 디스크 용량 확인

File Size	104,857,600
Physical Size	104,857,600
Start Cluster	72,881,246
Date Accessed	2026-02-11 오전 4:25:20
Date Created	2026-02-11 오전 4:25:15
Date Modified	2026-02-11 오전 4:25:20
Encrypted	False
Compressed	False
Actual File	True
Start Sector	583,049,968

SSD

■ [실습] TRIM

- 파일 생성 - 파일 삭제 - TRIM 명령 후 데이터 및 디스크 용량 변화

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
458145E000	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E010	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E020	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E030	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E040	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E050	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E060	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST

	사용 중인 공간:	367,075,577,856바이트	341GB
	여유 공간:	125,820,149,760바이트	117GB

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
458145E000	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E010	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E020	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E030	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E040	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E050	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST
458145E060	54	45	53	54	54	45	53	54	54	45	53	54	54	45	53	54	TESTTESTTESTTEST

	사용 중인 공간:	366,984,622,080바이트	341GB
	여유 공간:	125,911,105,536바이트	117GB

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
458145E000	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
458145E010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
458145E020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
458145E030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
458145E040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
458145E050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
458145E060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

	사용 중인 공간:	366,973,763,584바이트	341GB
	여유 공간:	125,921,964,032바이트	117GB

SD Card vs eMMC vs UFS

■ SD Card (Secure Digital Card)

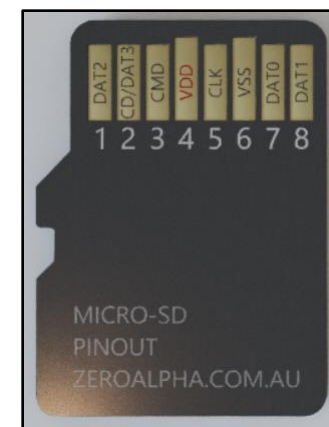
- 휴대용 저장장치에 사용하기 위해 개발한 범용 Flash 메모리 카드
- 호환성이 우수하여 카메라, PC 등 다양한 기기에서 사용
- 데이터 읽기 / 쓰기를 단방향으로 수행
- SD Card vs Micro SD Card[추가]
 - 크기가 다를 뿐 기능적으로 같은 SD 규격 사용
 - 등급과 인터페이스가 성능을 결정



[SD Card]



[SD Card]



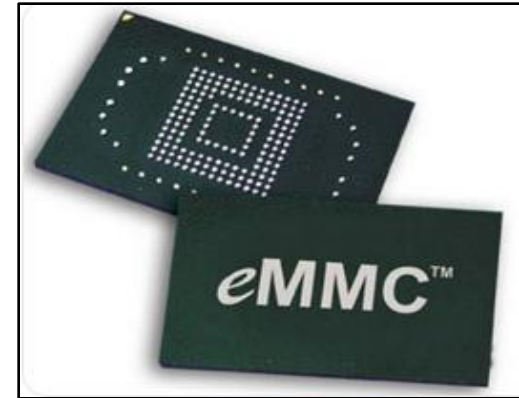
[Micro SD Card]

구분	SD Card	Micro SD Card
접점	9개	8개
크기 (가로 x 세로 x 두께)	32 x 24 x 2.1 (mm)	15 x 11 x 1.0 (mm)
주 사용처	카메라, 노트북 등	스마트폰, 액션캠, 드론 등

SD Card vs eMMC vs UFS

■ eMMC (Embedded MultiMedia Card)

- NAND 플래시와 컨트롤러가 결합되어 있는 Flash 메모리 카드
- 기기 메인보드에 내장되어 있어 탈착 불가
- 데이터 읽기 / 쓰기를 단방향으로 수행



[eMMC]

■ UFS (Universal Flash Storage)

- 최신 스마트폰 등 고성능 기기에 사용되는 차세대 Flash 메모리 카드
- 읽기 / 쓰기 동시 처리로 인한 전이중 통신 가능
- LVDS interface를 사용하여 낮은 전력 소모
 - LVDS (Low Voltage Differential Signaling)
 - 신호 전송 시 낮은 전압을 사용하여 전력 소모량을 줄이는 방법



[UFS]

SD Card vs eMMC vs UFS

■ SD Card / eMMC / UFS 비교[추가]

- SD Card, eMMC, UFS 모두 대용량, 고집적, 저가가 핵심 목표 → NAND Flash 사용
 - NAND Flash의 특징
 - NOR Flash와 달리 셀이 직렬로 연결되어 한 공간에 셀을 더 촘촘히 배치 가능 → 대용량, 고집적
 - 저장매체 자체가 작은 반도체 칩이며 공장에서 같은 제품을 대량으로 생산 → 저가

구분	SD Card	eMMC	UFS
성격	외장 저장매체	내장 저장장치	고성능 내장 저장장치
탈착	가능	불가	불가
데이터 처리	반이중	반이중	전이중
주 사용처	카메라, 드론, 블랙박스	저가 스마트폰, IoT	고급 모바일
성능	낮음	중간	높음

저장장치 인터페이스

정의

■ 개념 및 구성요소

- 저장장치와 호스트가 데이터와 명령을 주고 받기 위해 사용하는 인터페이스
- 디지털 포렌식에 과정에서 저장장치의 데이터 수집을 위해 인터페이스 식별이 필요
- Protocol
 - 데이터를 어떻게 주고 받을 지 약속한 소프트웨어 / 논리적 규칙
- Bus
 - 전기적 / 물리적 데이터의 전송 통로
- Form Factor
 - 장치의 크기나 배치 등 기계적인 규격

Protocol

■ ATA, AHCI, NVMe, SCSI[추가]

- ATA: Advanced Technology Attachment, HDD나 CD-ROM 등을 위한 저장장치 통신 프로토콜
 - 초기 PC 저장장치 표준, 현재는 SATA 환경에서 사용

버전	전송 방식	최대 속도	특징
ATA-1	PIO 0, 1, 2 Single-word DMA 0, 1, 2 / Multiword DMA 0	8.3MB/s	첫 표준 규격, 28 bits LBA
ATA-2	PIO 3, 4 / Multiword DMA 1, 2	16.6MB/s	E-IDE (Enhanced IDE)
ATA-3	-	16.6MB/s	S.M.A.R.T 지원, 보안성 개선
ATA / ATAPI-4 (Ultra ATA/33)	Ultra DMA 0, 1, 2	33MB/s	ATAPI 규격과 통합, HPA 도입
ATA / ATAPI-5 (Ultra ATA/66)	Ultra DMA 3, 4	66MB/s	40선 → 80선 케이블로 변경
ATA / ATAPI-6 (Ultra ATA/100)	Ultra DMA 5	100MB/s	48 bits LBA, DCO 도입
ATA / ATAPI-7 (Ultra ATA/133)	Ultra DMA 6	133MB/s (PATA) 150MB/s (SATA)	SATA 1.0 사용 시작

Protocol

■ ATA, AHCI, NVMe, SCSI[추가]

- ATA-1: PIO1, 2와 저속 DMA 전송을 사용해 최대 8.3MB/s 속도 제공
 - PIO (Programmed Input / Output): CPU가 직접 데이터 전송을 제어, 큰 CPU 부하
 - DMA (Direct Memory Access) : CPU의 개입 없이 장치가 직접 메모리로 전송, 작은 CPU 부하
- ATA-2: PIO 3, 4와 고속 Multiword DMA 1, 2를 추가해 속도를 16.6MB/s 까지 향상
 - PIO 3, 4는 ATA-1에서 쓰인 1, 2에 비해 I/O 사이클 타임을 줄여 단위 시간당 데이터 전송률을 향상
 - Multiword DMA: DMA 컨트롤러가 수 개의 word 단위로 전송
 - ATA-1 보다 속도 향상, CPU 부하를 줄여 E-IDE (Enhanced IDE)라고 명명

Protocol

- **ATA, AHCI, NVMe, SCSI[추가]**

- ATA-3: S.M.A.R.T 기능과 ATA Security Feature Set을 추가해 신뢰성과 보안성 강화
 - S.M.A.R.T: Self-Monitoring Analysis and Reporting Technology, 하드디스크가 자기 상태를 스스로 점검하고
이상을 보고하는 기능 → 고장 가능성을 사전에 감지
 - ATA Security Feature Set: 하드디스크 수준에서 잠금을 거는 기능
 - SECURITY SET PASSWORD: 디스크에 비밀번호를 설정
 - SECURITY FREEZE LOCK: 보안 설정 변경 명령 금지, 변경하려면 재부팅 필요
 - 전송 속도는 16.6 MB/s로 이전 버전과 동일

Protocol

■ ATA, AHCI, NVMe, SCSI[추가]

- ATA / ATAPI-4 (Ultra ATA / 33): Ultra DMA 도입으로 33MB/s까지 속도 향상
 - Ultra DMA: 고속 DMA 방식, CRC를 통한 전송 오류 검출이 가능하고 더블 엠티를 사용하여 데이터 전송
 - ATAPI 규격과 통합되어 광학 드라이브 등 디스크가 아닌 장치도 동일 인터페이스로 다루기 가능
 - HPA 도입: 디스크 일부 영역을 숨겨 시스템 복구 및 관리 기능 강화
- ATA / ATAPI-5 (Ultra ATA / 66): Ultra DMA 3, 4를 사용해 최대 66MB/s 속도 지원
 - 기존 40핀에서 80핀 케이블로 변경 → 신호선 사이에 접지선을 추가해 간섭 최소화 및 고속 전송 가능

Protocol

■ ATA, AHCI, NVMe, SCSI[추가]

- ATA / ATAPI-6 (Ultra ATA / 100): 48 bit LBA 도입 및 100MB/s로 속도 향상
 - 기존에는 28 bit LBA 사용 → 48 bit LBA 체계를 통해 대용량 디스크 지원
 - DCO 도입: 장치의 용량 및 기능을 제한하거나 숨기는 기능
- ATA / ATAPI-7 (Ultra ATA / 133): SATA 인터페이스 첫 도입
 - PATA 기준 133MB/s, SATA 기준 150MB/s 속도 지원
 - PATA는 신호 간섭, 타이밍 문제로 속도를 더 높이기엔 한계 존재
 - 병렬 인터페이스에서 직렬 인터페이스로 전환되는 계기

Protocol

- **ATA, AHCI, NVMe, SCSI[추가]**

- AHCI (Advanced Host Controller Interface)

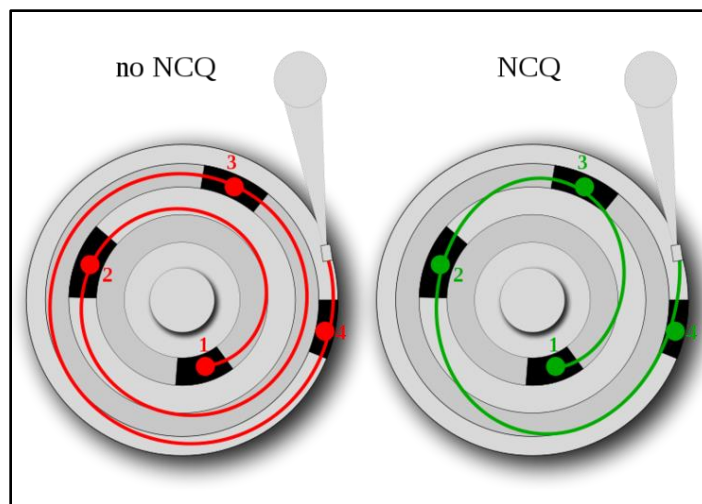
- ATA 명령을 SATA 저장장치에서 쓰기 위한 컨트롤러 인터페이스
 - NCQ (Native Command Queuing) 지원: 여러 개의 명령을 일시적으로 저장, 정렬하여 명령을 실행
 - Hot-Plugging: 전원이 켜진 상태에서 장치 연결 및 제거 가능
 - LPM (Link Power Management): 입출력이 없을 때 SATA 링크를 저전력 상태로 전환

Protocol

- ATA, AHCI, NVMe, SCSI[추가]

- AHCI (Advanced Host Controller Interface)

- NCQ (Native Command Queuing): 여러 입출력 명령어를 큐에 쌓아두고 최적 순서로 정렬하여 처리 (최대 32개)
 - HDD: 헤드 / 액추에이터를 많이 움직이지 않고 최적으로 처리 가능
 - SSD: HDD처럼 물리적인 이동은 없지만 큐의 명령들을 FTL 매핑 테이블로 변환해서 효율적 명령 처리



Protocol

- ATA, AHCI, NVMe, SCSI[추가]
 - AHCI (Advanced Host Controller Interface)
 - Command Queuing 정리

Command Queuing	적용 대상	최대 큐 개수	큐 당 최대 명령 수	특징
TCQ (Tagged Command Queuing)	PATA	1개	32개	매번 CPU가 디스크한테 DMA 설정 필요
NCQ (Native Command Queuing)	SATA	1개	32개	HBA가 DMA 설정
SCSI / SAS TCQ	SCSI / SAS	1개	256개	향상된 HBA가 DMA 설정
NVMe Multi Queue	NVMe (PCIe)	65,535개	65,535개	CPU 코어마다 독립 큐 사용, PCIe가 DMA 처리

Protocol

■ ATA, AHCI, NVMe, SCSI[추가]

■ AHCI (Advanced Host Controller Interface)

- Hot-Plugging: 전원이 켜진 상태에서 장치 연결 및 제거를 할 수 있는 기능
- 동작 원리 (연결)
 - 물리적 연결: 접지 핀 → 전원 핀 → 데이터 핀 순서로 연결 후 데이터 신호 안정되면 디스크 활성화
 - SATA PHY 링크: OOB 신호 교환 (COMRESET → COMINIT → COMWAKE)
 - AHCI 컨트롤러 감지: 포트 상태 변화 감지 후 인터럽트 발생
 - OS 장치 초기화: 포트 상태를 확인하여 디스크 특성 및 파티션을 읽고, 파일시스템 마운트

Protocol

■ ATA, AHCI, NVMe, SCSI[추가]

■ AHCI (Advanced Host Controller Interface)

- Hot-Plugging: 전원이 켜진 상태에서 장치 연결 및 제거를 할 수 있는 기능
- 동작 원리 (안전 제거)
 - OS 제거 요청: 사용자가 안전 제거를 요청하면, OS는 열린 파일과 파일시스템 메타데이터 정리
 - Cache Flush: OS가 디스크에 FLUSH CACHE 명령을 내리고, 디스크는 Write Cache의 데이터를 기록
 - AHCI 명령 정리: 진행 중인 입출력 종료, Command Queue 정리
 - 물리 제거: 장치를 분리하고 SATA 링크 다운
 - AHCI 감지: 포트 상태 감지 후 OS에 인터럽트를 발생
 - OS 디바이스 제거: 포트 확인 후 디바이스 제거 및 자원 해제

Protocol

- ATA, AHCI, NVMe, SCSI[추가]

- AHCI (Advanced Host Controller Interface)

- Hot-Plugging: 전원이 켜진 상태에서 장치 연결 및 제거를 할 수 있는 기능
- 동작 원리 (강제 제거)
 - 물리적 분리: 링크 끊김
 - SATA PHY: 링크 다운, COMRESET 실패
 - AHCI 감지: 포트 상태 감지 후 OS에 인터럽트 발생
 - OS 처리: 진행 중인 입출력 실패, Write Cache 손실 가능, 파일시스템 손상 가능

Protocol

- ATA, AHCI, NVMe, SCSI[추가]
 - AHCI (Advanced Host Controller Interface)
 - Hot-Plugging vs Hot-Swapping
 - Hot-Plugging은 연결, 제거, Hot-Swapping은 교체
 - 이외에도 전원을 끄지 않은 상태에서 연결하는 Hot-Insertion / 제거하는 Hot-Removal 개념 존재

항목	Hot-Plugging	Hot-Swapping
범위	연결, 제거	교체
핵심	전원을 켜 상태에서 장치 연결, 안전 제거	전원을 켜 상태에서 장치 교체
예시	HDD, SSD, USB 등 연결 / 제거	서버 RAID 디스크 교체, 전원 공급장치 교체 등

Protocol

- ATA, AHCI, NVMe, SCSI[추가]

- AHCI (Advanced Host Controller Interface)

- LPM (Link Power Management)

- 입출력이 없을 때 SATA 링크를 저전력 상태로 전환
- SATA 링크 전원 상태

상태	설명	복귀 속도	전력
Active	정상 동작	-	높음
Partial	얕은 절전	빠름	감소
Slumber	깊은 절전	느림	더 낮음
DevSleep	거의 OFF	가장 느림	최소

Protocol

■ ATA, AHCI, NVMe, SCSI[추가]

■ NVMe (Non-Volatile Memory express)

- PCIe 위에서 동작하는 SSD 전용 프로토콜
- 다중 큐 구조로 수만 개의 명령 대기열을 처리 (최대 65,535개)
- CPU 코어 별로 Queue를 가질 수 있어 Lock 경쟁이 없고 병렬 처리 가능

■ SCSI (Small Computer System Interface)

- 서버나 스토리지용으로 사용하는 프로토콜
- CDB (Command Descriptor Block) 명령 체계: 표준화된 형식으로 여러 장치를 일관된 형식으로 관리
- 명령 실행 후 Status 반환 구조: 오류 발생 시 상세한 원인 반환 → 명령 재시도 / 리맵 등 오류 복구 가능

Protocol

- ATA, AHCI, NVMe, SCSI[추가]
 - HDD / SSD 별 지원 Bus 및 프로토콜 정리

저장장치 종류	Bus	Protocol
SSD	SATA	ATA, AHCI
	PCIe	NVMe
	SAS	SCSI
	USB	ATA, AHCI, NVMe
HDD	PATA	ATA
	SATA	ATA, AHCI
	SAS	SCSI
	SCSI	SCSI
	USB	ATA

Bus

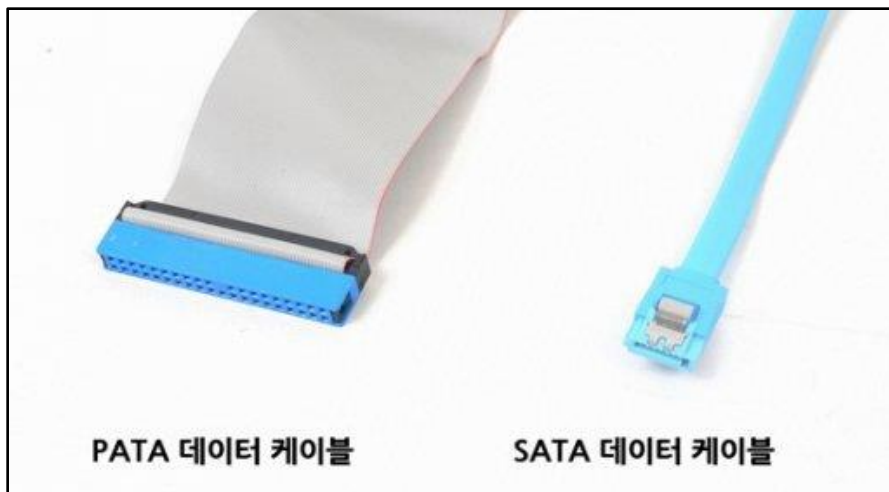
■ SATA, SAS, PCIe, USB[추가]

■ PATA (Parallel ATA)

- 병렬 방식으로 한 번에 16 bit 데이터 전송
- Hot Plugging 미지원
- 신호 간섭, 도착 타이밍 문제 등으로 낮은 전송 속도

■ SATA (Serial ATA)

- 현대 SSD / HDD 등 연결에 사용되는 인터페이스
- 직렬 방식으로 데이터를 하나의 선으로 보내 높은 전송 속도
- Hot Plugging 지원
- NCQ 지원



Bus

■ SATA, SAS, PCIe, USB[추가]

■ Parallel SCSI

- 과거 워크스테이션, 서버 등에서 사용되던 병렬 방식 인터페이스
- 하나의 Bus에 여러 장치 연결이 가능 → 신호 감쇠도 심해져 확장성에 한계 존재
- 병렬 전송 방식 → 신호 간섭 문제와 신호 도착 타이밍 문제 존재

■ SAS (Serial Attached SCSI)

- 서버용 고신뢰 저장장치에 사용되는 직렬 방식 인터페이스
- Point to Point 구조: 각 장치가 독립된 링크로 연결되므로 신호 간섭 감소 및 고속 전송 가능
- Expander를 사용하여 확장성 증대 → 대규모 서버 및 데이터센터 환경에 적합

Bus

■ SATA, SAS, PCIe, USB[추가]

■ PCI (Peripheral Component Interconnect)

- 메인보드 확장용으로 그래픽 카드, NIC 등 주변 장치 연결
 - Plug and Play: 주변 기기를 연결하면 별도의 수동 설정 없이 이용 가능한 기술

■ PCIe (Peripheral Component Interconnect express)

- CPU와 주변 장치를 연결하는 고속 직렬 확장 버스
 - NVMe SSD의 기반, x1, x4, x8, x16 등이 있으며 레인 수가 많을 수록 대역폭이 높음

버전	출시년도	x1 레인 속도	x16 레인 속도
PCIe 1.0	2003	250MB/s	4GB/s
PCIe 2.0	2007	500MB/s	8GB/s
PCIe 3.0	2010	1GB/s	16GB/s
PCIe 4.0	2017	2GB/s	32GB/s
PCIe 5.0	2019	4GB/s	64GB/s

Bus

■ 직렬 방식이 빠른 이유[추가]

■ 데이터 정렬

- 병렬 방식은 모든 신호가 같은 선에서 데이터가 동시에 도착해야 함
- 직렬 방식은 여러 통로에서 들어온 데이터를 Deskew 버퍼에 잠시 저장했다가 모든 레인의 타이밍을 맞춤
- 병렬로 모든 데이터의 도착을 기다리는 방식보다 버퍼에 잠시 저장 후 정렬하는 방식이 더 효율적

■ Point-to-Point

- 송수신 양측이 1:1로 직접 연결된 전용 통로를 사용
- 여러 장치가 하나의 통로를 공유하는 경우보다 신호 간섭이 적고 고속 전송 가능
- 각 링크가 하나의 비트를 전송하는 대신 클럭 수를 높여 매우 빠르게 전송

Bus

■ 직렬 방식이 빠른 이유[추가]

■ 차동 신호 방식

- 직렬 방식은 하나의 데이터 전송 시 두 개의 선을 사용
- 한쪽 선에서 원래 신호 (+)를 보내고 다른 선에서는 반대 신호 (-)를 동시에 전송
- 외부에서 노이즈가 들어오면 두 선 모두에 같은 노이즈가 추가
- 수신 측에서 두 선의 신호를 빼면 공통으로 들어있던 노이즈 제거가 가능

송신

- Line A: S
- Line B: $-S$



노이즈 유입

- Line A: $S + N$
- Line B: $-S + N$



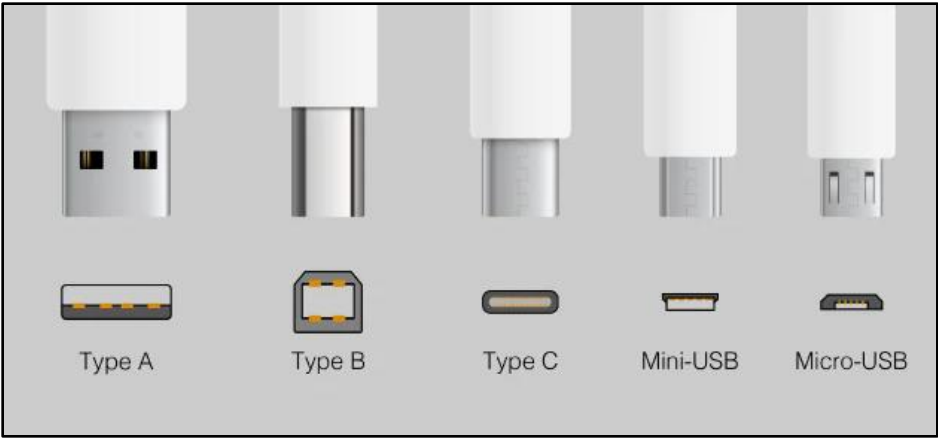
수신부의 계산

- $(S + N) - (-S + N)$

Bus

■ USB (Universal Serial Bus) Type 및 버전

- 키보드나 마우스 등 주변 기기를 연결하기 위한 범용 Bus
 - Type A, Type B, Mini-USB, Micro-USB의 경우 정해진 방향으로만 연결 가능
 - Type C는 대칭 구조로 양방향 연결이 가능
 - 최신 기기의 경우 Type C를 지원하는 경우가 많음



타입	모양	방향	사용 예시
Type A	직사각형	단방향	키보드, 마우스, USB 메모리 등
Type B	모서리가 깎인 사각형	단방향	프린터, 스캐너 등
Type C	타원형	양방향	최신 스마트폰, 노트북 등
Mini-USB	사다리꼴	단방향	옛 디지털 카메라, MP3 등
Micro-USB	사다리꼴	단방향	옛 안드로이드 기기, 소형 IoT 기기 등

Bus

■ USB 계층형 스타 토폴로지[추가]

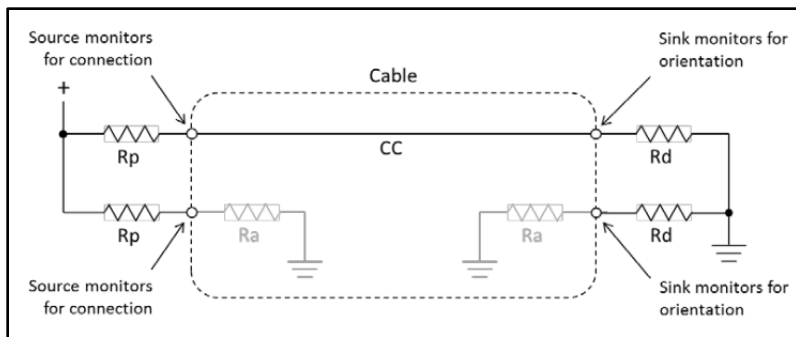
■ 계층형 스타 토폴로지

- 중심 허브에 여러 장치가 연결된 구조를 말하는 스타 토폴로지가 여러 계층으로 구현된 것
 - Tier 1: 최상위에는 Root Hub를 가진 호스트
 - Tier 2 ~ 6: host 본체 보트에 USB 허브를 꽂고 그 허브에 다시 허브를 꽂아 계층 생성
 - Tier 7: 장치 연결: 마우스, 키보드 등의 장치를 연결
- USB는 7 bit 주소 체계를 사용해 연결된 기기에 주소 할당
 - 0번 주소는 처음 기기 연결 시 주소 할당 전 임시 대기용으로 사용 → 최대 127개 기기 연결 가능

Bus

■ USB Type-C 호스트 / 디바이스 역할 구분[추가]

- 호스트: USB 통신을 주도하면서 전원을 공급하고, 장치를 탐색·설정하는 쪽
- 디바이스: 호스트가 보내는 요청에 응답해 데이터를 주고받는 주변 장치
- USB Type-C 연결 단자에는 CC (Configuration Channel)핀이 두 개 존재
- 기기가 연결되었을 때 CC 핀에 걸리는 저항을 통해서 호스트 / 디바이스 역할 구분
 - Rp (Pull-up Resistor): 전압을 올리는 저항 / Rd (Pull-down Resistor): 전압을 내리는 저항
 - 전압이 높은 곳에서 낮은 곳으로 흐르도록 설정하여 호스트 측에서 디바이스로 request 보내는 구조



Bus

■ USB 버전[추가]

- 버전이 올라갈수록 최고 전송 속도가 향상
- 3.0 버전 이후로 'x숫자' 형식으로 레인 수 표기
- 소비자를 위해 USB-IF는 마케팅명을 제작
 - USB-IF: USB Implementers Forum,
표준을 만들고 관리하는 비영리 단체
- 새 기술이 나올 때 마다 이전 버전 이름까지 업데이트



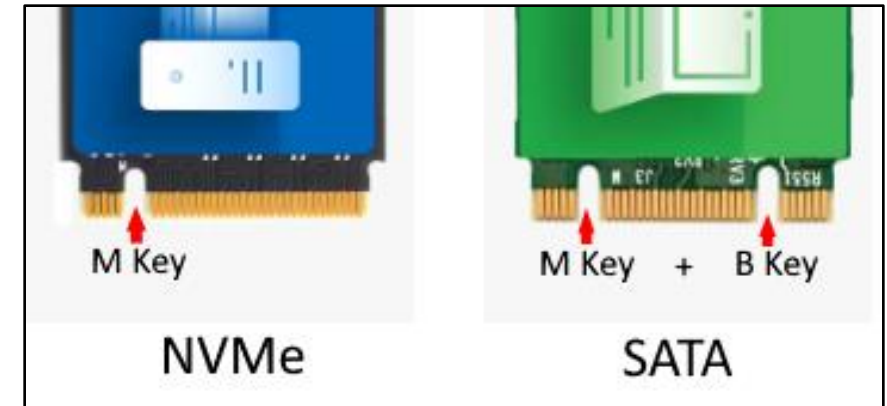
USB 버전	전송 속도	지원 인터페이스	마케팅명
1.0	1.5Mbps	USB-A, USB-B	Basic-Speed USB
1.1	12Mbps	USB-A, USB-B	
2.0	480Mbps	USB-A, USB-B	High-Speed USB
3.0	5Gbps	USB-A, USB-B, USB-C	USB 5Gbps
3.1 Gen 1			
3.2 Gen 1x1			
3.1	10Gbps	USB-C	USB 10Gbps
3.1 Gen 2			
3.2 Gen 1x2			
3.2 Gen 2x1			
3.2 Gen 2x2	20Gbps	USB-C	USB 20Gbps
4.0 Gen 3x1	20Gbps	USB-C	
4.0 Gen 3x2	40Gbps	USB-C	USB 40Gbps

Form Factor

■ M.2, U.2, U.3, PCIe 슬롯[추가]

■ M.2

- 메인보드에 SSD를 직접 꽂는 카드 형태 폼팩터
- 여러 Bus를 공통으로 쓰기 위해서 설계
- 노트북, 데스크탑 등 흔하게 사용
- SATA, NVMe (PCIe) 지원
 - M key: NVMe / B+M key: SATA
 - 전기 신호에 차이가 존재
 - 키에 차이를 둬서 오용, 사용자 실수 등을 물리적으로 차단



Form Factor

■ M.2, U.2, U.3, PCIe 슬롯[추가]

■ M.2

• M.2 홈 (Key)

- B+M Key는 Key B와 Key M 홈에 어디든 장착 가능한 호환성을 목적으로 설계
- B Key 쪽 홈을 파내는 과정에서 PCIe의 3번, 4번 레인을 담당하는 핀들이 제거
- 실질적으로 PCIe x2 까지만 지원

Key ID	홈 위치	버스	주 용도
B	12-19	SATA / PCIe x1, x2 / USB 2.0, 3.0	SATA SSD
M	59-66	SATA / PCIe x1, x2, x4	NVMe SSD (고성능)
B+M	12-19, 59-66	SATA / PCIe x1, x2	SATA SSD (호환용)

Form Factor

■ M.2, U.2, U.3, PCIe 슬롯[추가]

■ M.2

• M.2 크기

- 앞 2자리는 폭, 뒤 2자리 이상은 길이 의미
- 폭이 22mm인 경우가 많지만 16mm, 30mm 등의 다양한 규격도 존재
- 폼팩터의 크기가 클수록 더 많은 NAND 칩 등을 탑재할 수 있기에 고용량 제품에 유리

크기	2230	2242	2260	2280	22110
폭	22 mm	22 mm	22 mm	22 mm	22 mm
길이	30 mm	42 mm	60 mm	80 mm	110 mm
주 용도	초소형 기기·태블릿용	소형 노트북·미니 PC용	구형 노트북·소형 PC용	데스크탑·노트북용	고용량·서버용

Form Factor

■ M.2, U.2, U.3, PCIe 슬롯[추가]

■ U.2

- 기업용 서버에서 NVMe SSD를 연결하기 위해 설계된 인터페이스
- 4개의 PCIe 레인 / 2개의 SAS 레인 / 1개의 SATA 레인 지원
- 어떤 컨트롤러에 연결 되어 있느냐에 따라 인터페이스 결정
 - ex. 백플레인이 PCIe에 연결 → NVMe만 동작
- 충분한 전력 공급과 발열 관리가 가능해 서버용으로 적합
- 키 길이에 차이를 두어 Hot Plugging 지원
 - 꽂을 때: 접지 핀 → 전원 핀 (ON) → 데이터 핀
 - 뺄 때: 데이터 핀 → 전원 핀 (OFF) → 접지 핀



Form Factor

■ M.2, U.2, U.3, PCIe 슬롯[추가]

■ U.3

- 하나의 슬롯에 SATA, SAS, NVMe를 지원하는 통합 폼팩터
 - U.2의 확장 형태로 유연성이 높아서 서버, 스토리지 구성에 사용
 - Tri-Mode 컨트롤러가 동적으로 프로토콜을 식별 및 연결 → PCIe / SAS / SATA 모두 사용 가능

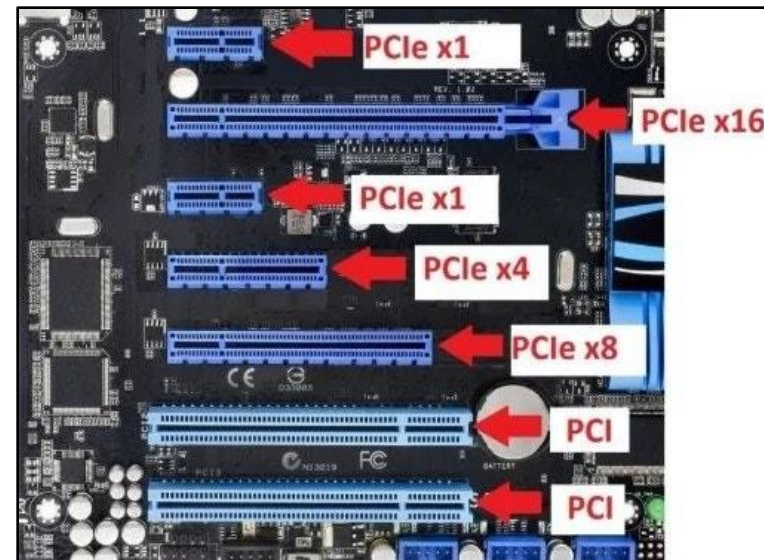
특징	U.2	U.3
인터페이스	NVMe (PCIe), SATA, SAS	NVMe(PCIe), SAS, SATA (동시 지원)
속도	주로 PCIe 3.0 기반 (최대 약 32Gbps, x4 레인)	주로 PCIe 4.0 기반 (최대 약 64Gbps, x4 레인)
핀	프로토콜별 핀 분리	공통 핀
백플레인	프로토콜별로 백플레인 분리	유니버설 백플레인 1개
컨트롤러	프로토콜별로 컨트롤러 분리	Tri-Mode 컨트롤러 1개
호환	상위 호환 (U.2 → U.3) 불가	하위 호환 (U.3 → U.2) 가능
Hot Plugging	지원	지원

Form Factor

- M.2, U.2, U.3, PCIe 슬롯 [추가]

- PCIe 슬롯: 메인보드의 PCIe 슬롯에 장착하는 확장형 방식

- 그래픽 카드, 네트워크 카드 연결 등에 사용
- CPU와 직접 연결되므로 지연시간이 낮음
- 레인 수 (x1 / x4 / x8 / x16)가 클수록 대역폭 증가
 - ex. 그래픽 카드 등 고성능을 요구하는 경우 x16 사용



이미징

정의

■ 개념

- 원본 저장장치의 데이터를 변경 없이 복제해서 사본을 생성하는 것
- 주로 법적 증거로 쓰기 위한 목적
 - Write Blocker를 사용하여 쓰기 차단
 - 해시 값을 확인해서 변조 여부 확인



■ 물리 이미지 vs 논리 이미지

- 물리 이미지: 디스크 전체를 복제하여 비할당 영역, HPA 영역 등 모든 정보를 포함
- 논리 이미지: 파일 시스템 단위로 복제하여 비할당 영역, HPA 영역 정보 미포함

물리 이미지 포맷[추가]

■ RAW (dd, 001)

- 아무런 가공도 하지 않은 순수 비트스트림 이미지
- 헤더, 메타데이터 등의 설명 정보 부재
- dd: 리눅스에서 dd 명령어로 생성한 raw 이미지
- 001, 002 등: 이미징 도구 (FTK Imager 등)에서 RAW 이미지를 생성할 때, 용량을 분할하여 저장할 경우 붙는 확장자
- 포맷 자체에 압축 기능 부재
 - 압축을 위해서는 압축 대상에 대한 메타데이터와 같은 정보 필요
 - 소스 그대로의 비트스트림 데이터이기 때문에 합의된 압축 포맷 부재
 - 장시간 압축 시 전원 차단과 같은 외부 요인에 의해 데이터 손상 가능

물리 이미지 포맷[추가]

■ E01, Ex01

- EnCase에서 증거 관리용으로 만든 이미지 포맷
- RAW 데이터에 케이스 정보, 해시 값 등을 포함한 형태
- 압축 기능 지원, 데이터 분할 저장 가능
- E01: 오래된 기본 포맷
 - MD5, SHA-1 등 전통적 해시 지원 → 충돌쌍 등 취약점 존재
 - 32 bit 기반 설계로 호환성 좋음
- Ex01: E01의 확장된 최신 포맷
 - SHA-256, SHA-512 등 최신 해시 지원 → 긴 해시 길이로 현재까지 충돌쌍 발견 X
 - 64 bit 기반 설계로 대용량 데이터 다루기 용이



논리 이미지 포맷[추가]

■ L01, Lx01

- EnCase에서 파일 / 폴더 단위 수집을 위해 만든 이미징 포맷
- 파일 / 폴더 마다 개별적으로 해시 처리
 - 구체적인 파일 / 폴더 대상 무결성 검증이 용이
- 삭제된 파일, 비할당 영역 정보는 수집 X
- L01: 구형 논리 이미지 포맷, 대용량, 다수 파일 처리에 한계 존재
 - 하나의 논리 컨테이너 안에 모든 파일을 순차적으로 저장 → 파일 수가 많아지면 탐색 속도 저하
- Lx01: L01의 확장 개선 버전, 대용량, 다수 파일 처리에 적합
 - 파일이 많아질 경우에도 여러 개의 섹션 구조로 인덱싱 속도 향상



논리 이미지 포맷[추가]

■ AD1

- FTK Imager에서 주로 사용되는 논리 이미지 포맷
 - 디스크 전체가 아닌 파일 / 폴더 단위 수집
- 압축 / 암호화 기능 지원
- 기본은 .ad1 확장자 → 분할 시 .ad2, .ad3로 이어짐
- FTK에서 최적화 되어 있어서 타 도구에서 호환성이 낮을 수 있음



[실습] EnCase[추가]

■ EnCase 설명 및 기능 소개

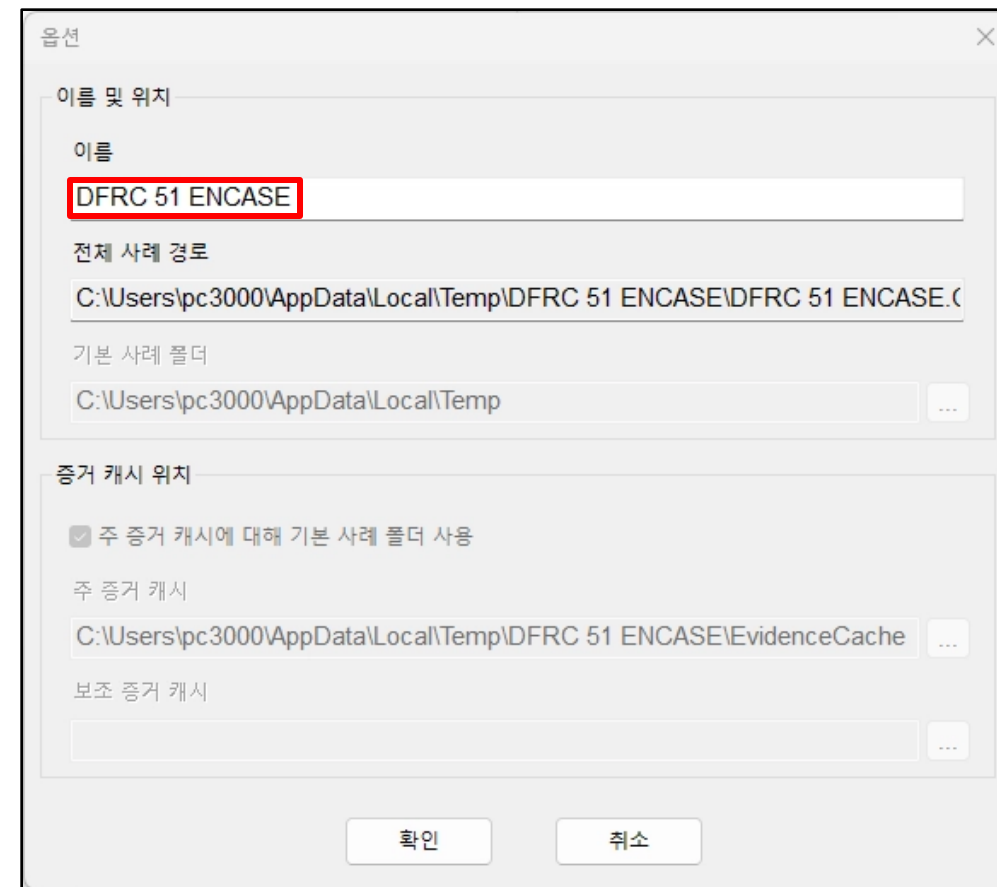
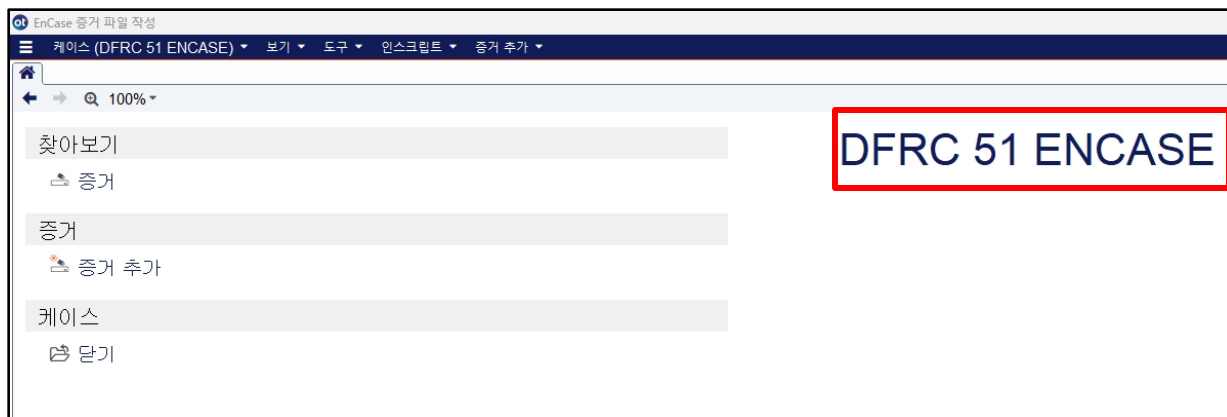
- 법 집행 기관 및 기업에서 디지털 증거를 수집, 분석, 보고하는데 사용하는 소프트웨어
- OpenText (구 Guidance Software)에서 개발
- 주요 기능
 - 증거 수집: 디스크, USB 등 저장 장치의 이미지 생성 및 해싱 → 실습 진행
 - 삭제 파일 복구: 비할당 영역 등에서 카빙을 통해 삭제된 파일 복구
 - 아티팩트 분석: 레지스트리, 이벤트 로그 등 아티팩트 분석
 - 보고서 생성: 생성 / 분석된 증거를 통합하여 최종 보고서 생성

[실습] EnCase[추가]

■ EnCase 설명 및 기능 소개

■ 케이스 생성

- 사건 별로 케이스 생성 가능
- 다양한 사건 및 증거 관리에 유용
- 증거 찾아보기, 증거 추가, 케이스 닫기 기능 제공



[실습] EnCase[추가]

■ EnCase 설명 및 기능 소개

■ 증거 추가

- 로컬 디바이스: 현재 장치에 있는 드라이브 자체를 증거로 추가
- 증거 파일: EnCase에서 만든 Ex01 / Lx01 같은 EnCase 포맷 이미지 등 추가 시 사용
- 원시 이미지: .raw, .dd 등 압축이나 메타데이터가 없는 순수 덤프 이미지 추가 시 사용

증거 추가

추가

-  로컬 디바이스
-  증거 파일
-  원시 이미지

[실습] EnCase[추가]

■ EnCase 설명 및 기능 소개

■ 이미지 생성

- EnCase에서 만든 케이스에 로컬 디바이스 로드 후 이미징 할 로컬 장치 선택
- 물리 / 논리 드라이브 선택, 특정 장치 선택 기능 → 선별 수집 가능
- Preview: 초기 수사 단계에서 해당 증거를 이미지화 할 가치가 있는지 판단하는 미리 보기 기능

■ 해당 로컬 디바이스 로드 시 논리 이미지 형성은 가능하지만 물리 이미지 형성은 불가

로컬 장치									
선택됨 2/9 편집									
	이름	레이블	액세스	섹터	크기	쓰기 금지되었습니다	파일 시스템을 읽습니다	링크 파일 구분 분석	DCO 있음
☐ 1	0	Samsung	ASPI	500,118,192	238.5 GB	•	☒	☒	
☐ 2	C	NTFS	윈도우즈	498,376,703	237.6 GB		☒	☒	
☒ 3	2	SanDisk	ASPI	61,341,696	29.3 GB		☒	☒	
☒ 4	D	XWAYS	윈도우즈	61,339,616	29.2 GB		☒	☒	
☐ 5	E	Msft	ASPI	954,388	1.8 GB		☒	☒	
☐ 6	C:\	Preview	Directory	498,376,703	237.6 GB		☒	☒	
☐ 7	D:\	Preview	Directory	61,339,616	29.2 GB		☒	☒	
☐ 8	E:\	Preview	Directory	954,388	1.8 GB		☒	☒	
☐ 9	램		메모리		9.7 GB				

[실습] EnCase[추가]

■ EnCase 설명 및 기능 소개

■ 증거 파일 작성: 물리 이미지 생성 기능

- 위치 탭: 증거 파일 저장 경로, 이름, 증거 번호, 케이스 번호, 조사자 이름 등 메타데이터 입력
- 포맷 탭: 증거 파일 형식, 압축 활성화 여부, 해시 종류 선택, 파일 세그먼트 크기 선택



2 디바이스의 증거 파일 작성

위치 포맷 고급

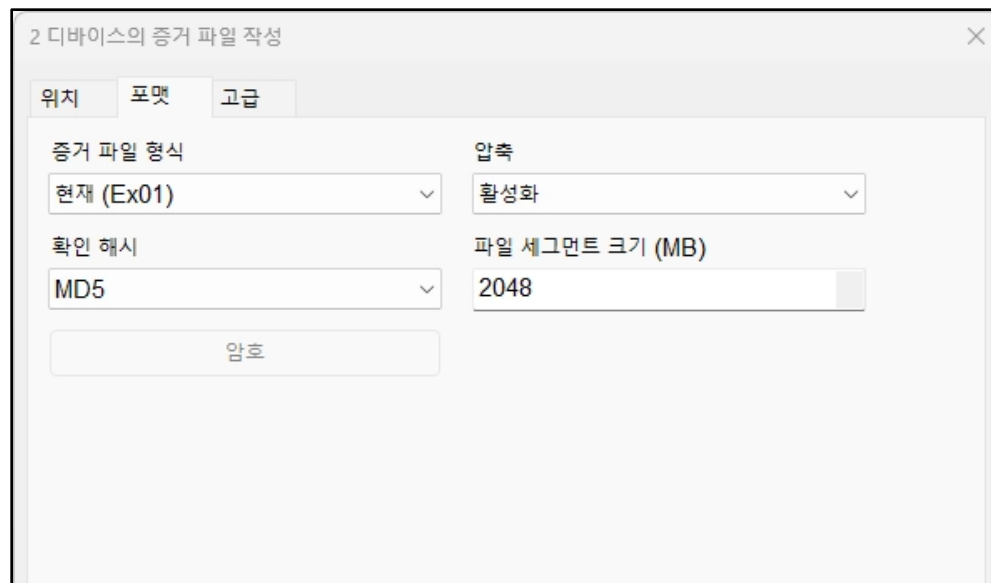
이름: DFRC 51 증거 번호: 1

케이스 번호: 1 조사자 이름: 51기

노트: Encase 실습

☐ 증거 파일 작성을 재시작합니다 ☐ 원격 수집

출력 경로: C:\Program Files\EnCase23\DFRC 51.Ex01



2 디바이스의 증거 파일 작성

위치 포맷 고급

증거 파일 형식: 현재 (Ex01) 압축: 활성화

확인 해시: MD5 파일 세그먼트 크기 (MB): 2048

암호

[실습] EnCase[추가]

■ EnCase 설명 및 기능 소개

■ EnCase 물리 이미지 포맷 정리

- 증거 파일 작성 탭에서
증거 파일 형식 및 해시 설정
- 파일 세그먼트 크기 설정 가능
 - 이미지가 클 경우 자동 분할



증거 파일 형식	확인 가능 해시							암호 기능 제공 여부
	없음	MD5	SHA-1	MD5 및 SHA-1	SHA-256	SHA-512	모두	
레거시 (E01)	O	O	O	O	X	X	X	O
현재 (Ex01)	O	O	O	X	O	O	O	X

[실습] EnCase[추가]

■ EnCase 설명 및 기능 소개

■ 논리적 증거 파일 작성 (L01 / Lx01 / AFF4-L)

- 위치 탭: 증거 파일 저장 경로, 이름, 증거 번호, 케이스 번호, 조사자 이름 등 메타데이터 입력
- 포맷 탭: 증거 파일 형식, 압축 활성화 여부, 해시 종류 선택, 파일 세그먼트 크기 선택



논리적 증거 파일을 작성합니다

위치 논리적 포맷

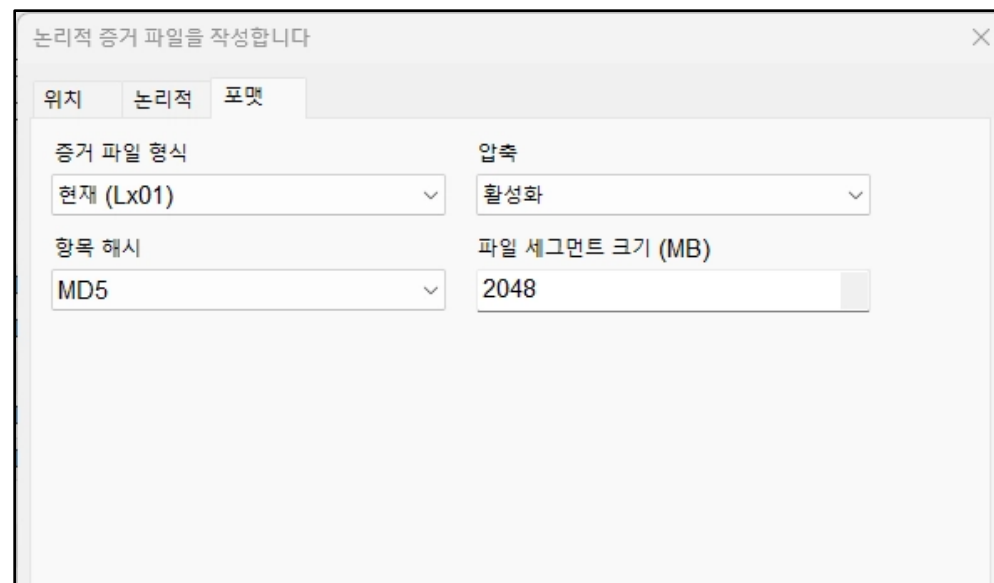
이름: dfrc 실습 증거 번호: 1

케이스 번호: 1 조사자 이름: 51기

노트:

☐ 기존 증거 파일에 추가합니다

출력 경로: C:\Program Files\EnCase23\dfrc 실습.Lx01



논리적 증거 파일을 작성합니다

위치 논리적 포맷

증거 파일 형식: 현재 (Lx01) 압축: 활성화

항목 해시: MD5 파일 세그먼트 크기 (MB): 2048

[실습] EnCase[추가]

■ EnCase 설명 및 기능 소개

■ EnCase 논리 이미지 포맷 정리

- 논리적 증거 파일 작성 탭에서
증거 파일 형식 및 해시 설정
- 파일 세그먼트 크기 설정 가능
 - 이미지가 클 경우
자동 분할

논리적 증거 파일을 작성합니다

위치 논리적 포맷

증거 파일 형식: 레거시 (L01)

압축: 활성화

항목 해시: MD5

파일 세그먼트 크기 (MB): 2048

논리적 증거 파일을 작성합니다

위치 논리적 포맷

증거 파일 형식: 현재 (Lx01)

압축: 활성화

항목 해시: MD5

파일 세그먼트 크기 (MB): 2048

논리적 증거 파일을 작성합니다

위치 논리적 포맷

증거 파일 형식: 레거시 (L01)

압축: 활성화

항목 해시: MD5

파일 세그먼트 크기 (MB): 2048

암호화

논리적 증거 파일을 작성합니다

위치 논리적 포맷

증거 파일 형식: 현재 (Lx01)

압축: 활성화

항목 해시: MD5

파일 세그먼트 크기 (MB): 2048

암호화

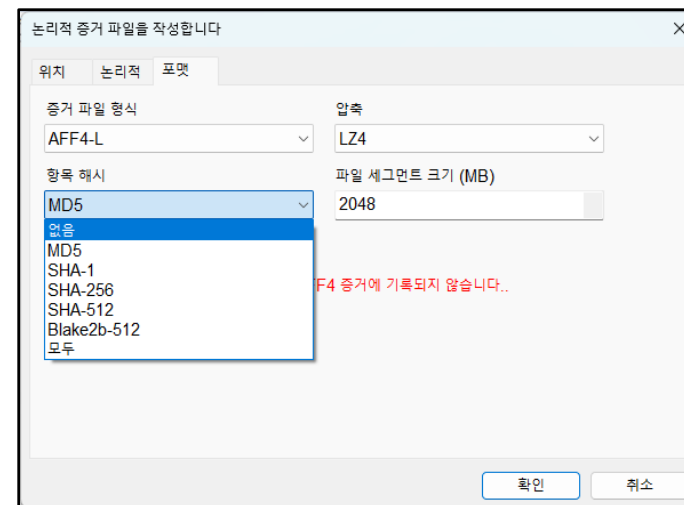
증거 파일 형식	확인 가능 해시					암호 기능 제공 여부
	없음	MD5	SHA-1	MD5 및 SHA-1	모두	
레거시 (L01)	○	○	X	X	X	X
현재 (Lx01)	○	○	X	X	X	○

[실습] EnCase[추가]

■ EnCase 설명 및 기능 소개

■ AFF4-L (Advanced Forensic Format 4)

- 2009년 제안된 논리 이미지 포맷
- 블록 해싱을 통해 기존 선형 해싱 방식보다 빠른 무결성 검증
 - 블록 해싱: 블록 단위로 나눠서 해싱을 하는 것
 - 선형 해싱: 데이터를 처음부터 끝까지 읽은 후 해싱을 하는 것
 - EnCase 기준 MD5, SHA-1, SHA-256, SHA-512, Blake2b-512 해시 지원
- L01, Lx01은 EnCase 사의 독점 포맷 → AFF4-L은 오픈 포맷 형식으로 벤더 중립적



[실습] EnCase[추가]

■ EnCase 설명 및 기능 소개

■ 이미지 생성 후

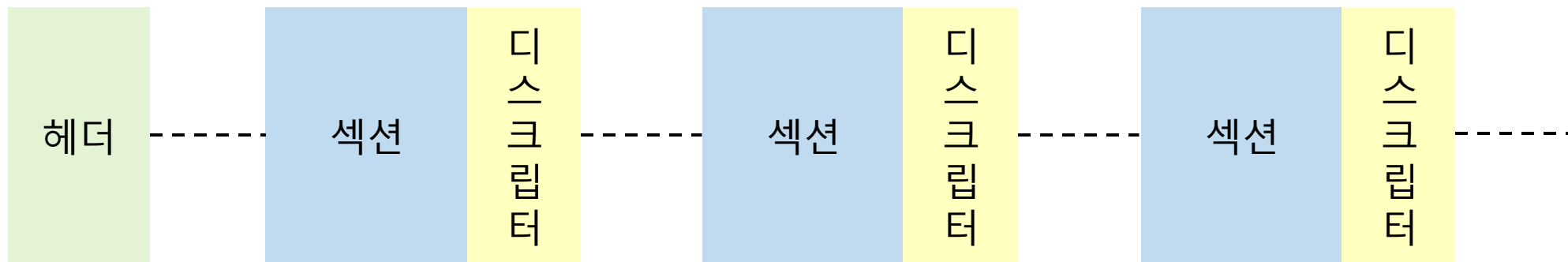
- 사용자가 지정한 경로로 이동하면 생성된 이미지를 확인 가능
- 설정한 파일 세그먼트 크기보다 이미지가 큰 경우, Ex02, Ex03 등으로 자동 분할

 DFRC 51	2026-02-18 오후 2:29	EnCase 증거 파일
 dfrc 실습	2026-02-18 오후 2:29	EnCase 논리적 증거 파일
 encase dfrc	2026-02-18 오후 2:21	EnCase 증거 파일
 encase dfrc.Ex02	2026-02-18 오후 2:23	EX02 파일
 encase dfrc.Ex03	2026-02-18 오후 2:25	EX03 파일
 encase dfrc.Ex04	2026-02-18 오후 2:29	EX04 파일

[실습] EnCase[추가]

■ Ex01, Lx01 구조

- 헤더와 여러 개의 섹션으로 구성
 - 헤더는 시그니처와 버전 정보, 압축 방식 등으로 구성 (32 Bytes)
 - 헤더 이후 여러 개의 섹션이 등장하며 각 섹션 끝에 64 Bytes의 섹션 디스크립터가 존재
 - 섹션 디스크립터 값에 따라서 디바이스 정보, 케이스 정보, MD5 / SHA-1 해시 값 등이 포함
 - 마지막 분할 파일인 경우 섹션 디스크립터 Section Type이 Done으로 설정
 - 다음 분할 파일 존재 시 섹션 디스크립터 Section Type이 Next로 설정



[실습] EnCase[추가]

■ Ex01 헤더 필드 분석

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	45	56	46	32	0D	0A	81	00	02	01	01	00	01	00	00	00	EVF2.....
00000010	A1	23	86	8F	A5	2A	1F	CB	8C	39	ED	93	3C	91	1A	66	i#†.¥*.ĚŒ9í“<‘.f
00000020	78	DA	35	8E	41	0A	C2	30	10	45	FF	36	3D	85	17	50	xÚ5ŽA.Â0.Eÿ6=...P

Offset	Field Name	Size	Description / Value
0x00	Signature	8	Signature / EVF2WrWnWx81Wx00, 공통 시그니처
0x08	Major Version	1	Major Version / 2, 2.1 버전에서 2를 의미
0x09	Minor Version	1	Minor Version / 1, 2.1 버전에서 1을 의미
0x0A	Compression Method	2	Compression Method / 1, COMPRESSION_LZ, zlib / deflate (0: No Compression, 2: bzip2)
0x0C	Segment file number	4	분할 저장된 파일 순서/ 1, Ex01에서 1을 의미 (Ex02인 경우 2)
0x10	Segment file set identifier	16	GUID, 모든 Ex01, Ex02는 같은 값을 가짐 / 0x661A913C93ED398CCB1F2AA58F8623A1
0x20	Section Area	-	헤더 이후 실제 섹션이 시작되는 부분 78 DA는 zlib / deflate로 압축된 데이터를 의미

[실습] EnCase[추가]

■ Ex01 섹션 디스크립터 분석

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
0226FDE0	0F	00	00	00	00	00	00	00	A0	FD	26	02	00	00	00	00	 ý&.....
0226FDF0	00	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
0226FE00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0226FE10	00	00	00	00	00	00	00	00	00	00	00	00	15	02	75	67	ug

Offset	Field Name	Size	Description / Value
0x00	Section Type	4	섹션의 종류 식별 / 0x0F: Done
0x04	Data Flags	4	데이터의 암호 또는 해시 여부 / 0x00: No Flag
0x08	Previous Section Offset	8	이전 섹션 오프셋 / 0x0226FDA0
0x10	Data Size	8	데이터 크기 / 0x0
0x18	Descriptor Size	4	16 Byte 섹션 디스크립터 크기 / 0x40
0x1B	Padding Size	4	정렬 목적의 패딩 크기 / 0x0
0x20	Data Integrity Hash	16	해시 설정: MD5 포함, 암호화: 암호값의 해시 값 포함
0x30	Padding	12	예약 영역 / 0
0x3C	Checksum	4	데이터 무결성 검사용 / 0x67750215

[실습] EnCase[추가]

■ Lx01 헤더 필드 분석

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	4C	45	46	32	0D	0A	81	00	02	01	01	00	01	00	00	00	LEF2.....
00000010	53	E6	DA	99	69	E9	CD	CF	B3	E7	70	27	D7	27	11	B1	SæÚ™iéİİ³çp'x'.±
00000020	78	DA	3D	8E	5D	0E	41	31	10	46	BF	D7	E9	2A	AC	40	xÚ=Ž].A1.F¿xé*-@

Offset	Field Name	Size	Description / Value
0x00	Signature	8	Signature / LEF2WrWnWx81Wx00, 공통 시그니처
0x08	Major Version	1	Major Version / 2, 2.1 버전에서 2를 의미
0x09	Minor Version	1	Minor Version / 1, 2.1 버전에서 1을 의미
0x0A	Compression Method	2	Compression Method / 1, COMPRESSION_LZ, zlib / deflate (0: No Compression, 2: bzip2)
0x0C	Segment file number	4	분할된 파일의 순번 / 1, Lx01에서 1을 의미 (Lx02인 경우 2)
0x10	Segment file set identifier	16	GUID, 모든 Lx01, Lx02는 같은 값을 가짐 / 0xB11127D72770E7B3CFCDE96999DAE653
0x20	Section Area	-	헤더 이후 실제 섹션이 시작되는 부분 78 DA는 zlib / deflate로 압축된 데이터를 의미

[실습] EnCase[추가]

■ Lx01 섹션 디스크립터 분석

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
02763030	0F	00	00	00	00	00	00	00	F0	2F	76	02	00	00	00	00	ð/v.....
02763040	00	00	00	00	00	00	00	00	40	00	00	00	00	00	00	00	@.....
02763050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
02763060	00	00	00	00	00	00	00	00	00	00	00	00	E7	01	4B	5E	ç.K^

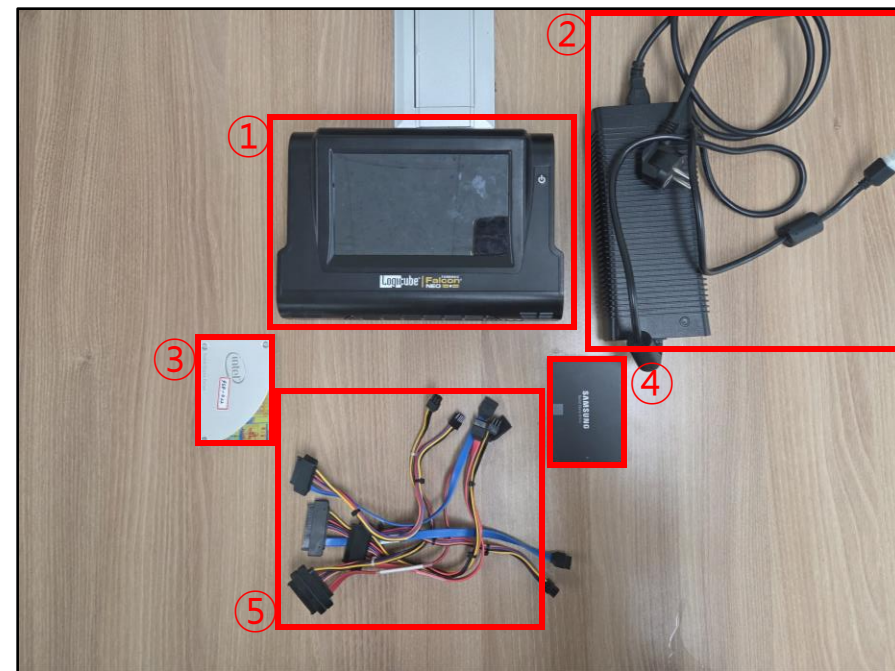
Offset	Field Name	Size	Description / Value
0x00	Section Type	4	섹션의 종류 식별 / 0x0F: Done
0x04	Data Flags	4	데이터의 암호 또는 해시 여부 / 0x00: No Flag
0x08	Previous Section Offset	8	이전 섹션 오프셋 / 0x02762FF0
0x10	Data Size	8	데이터 크기 / 0x0
0x18	Descriptor Size	4	섹션 디스크립터 크기 / 0x40
0x1B	Padding Size	4	16 Byte 정렬 목적의 패딩 크기 / 0x0
0x20	Data Integrity Hash	16	해시 설정: MD5 포함, 암호화: 암호값의 해시 값 포함
0x30	Padding	12	예약 영역 / 0
0x3C	Checksum	4	데이터 무결성 검사용 / 0x5E4B01E7

PC 데이터 수집

[실습] 디스크 덤프 및 분석

■ Falcon Neo 준비

- ① 메인 장비
- ② 파워 케이블
- ③ 원본 저장장치
- ④ 복제될 저장장치
- ⑤ 저장장치 연결 케이블



[실습] 디스크 덤프 및 분석

■ 장비와 저장장치 연결

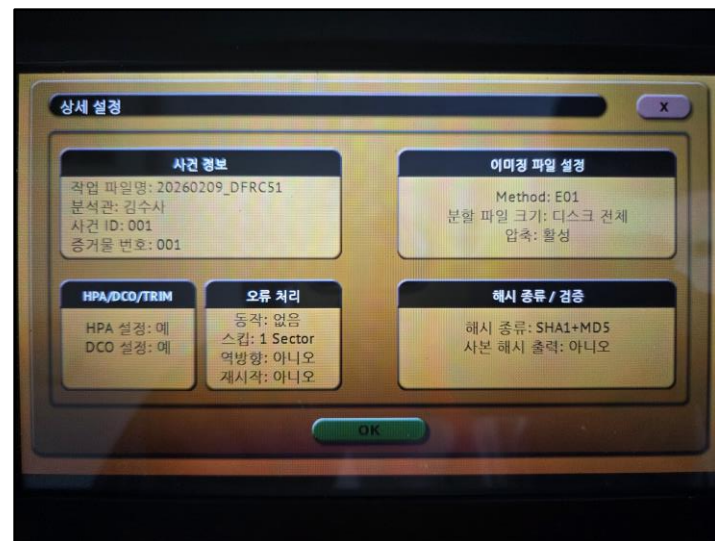
- 좌측 Source 포트에 원본 저장장치 연결
- 우측 Destination 포트에 복제될 저장장치 연결
- 주의
 - 원본 저장장치가 우측에 연결될 경우 데이터 손실
 - SAS / SATA의 경우 각 포트에 맞는 6핀 전원 포트에 연결



[실습] 디스크 덤프 및 분석

■ 이미징 방식 설정

- 사건 정보: 사건 메타 데이터 (파일명, 분석관, 사건 번호, 증거 번호) 입력
- 이미징 파일 설정: 이미징 포맷 (E01, Raw 등), 분할 크기, 압축 등의 이미징 방식 설정
- HPA / DCO / TRIM: HPA / DCO 영역까지 획득할 것인지, 이미징 후 TRIM을 수행할지 설정
- 해시 종류 / 검증: 이미징 후 계산할 해시 알고리즘 선택
- 오류 처리: 이미징 과정에서 발생하는 오류 처리를 위한 설정
 - 기본: Skip (1 섹터 단위로 건너뛰기)
 - 역방향: 배드 섹터를 건너뛴 후 뒤에서부터 다시 읽기
 - 재시작: 처음부터 다시 시작하지 않고 중단된 지점부터 다시 이미징



[실습] 디스크 덤프 및 분석

■ 이미징

- 작업 종류: [디스크 → 파일] (기본값) / [디스크 → 디스크] / [파일 → 파일] 등을 선택
- 원본: 연결된 원본 저장장치 선택
- 사본: 연결된 복제될 저장장치 선택
- 우측 하단의 시작 버튼으로 이미징 시작



[실습] 디스크 덤프 및 분석

■ 이미징

- 원본의 해시 (SHA1, MD5)가 복제된 이미지 (사본)의 해시와 동일함을 확인
 - 원본과 복제된 이미지 (사본)가 변조되지 않음을 입증 가능 (무결성 유지)

Hash Information	
LBA Count:	468862128
Sector Size:	512
Hash Type:	SHA-1
Source Hash:	3b05693256fdb5da70fb4198df6d39e49784d9d8
Secondary Hash Type:	MD5
Secondary Source Hash:	7039515afaef479e0dff91a8b8aabc9

Case Information	
Case/File Name:	20260209_DFRC51
Case ID:	001
Examiner:	김수사
Evidence ID:	001
Case Notes:	

[원본 디스크]

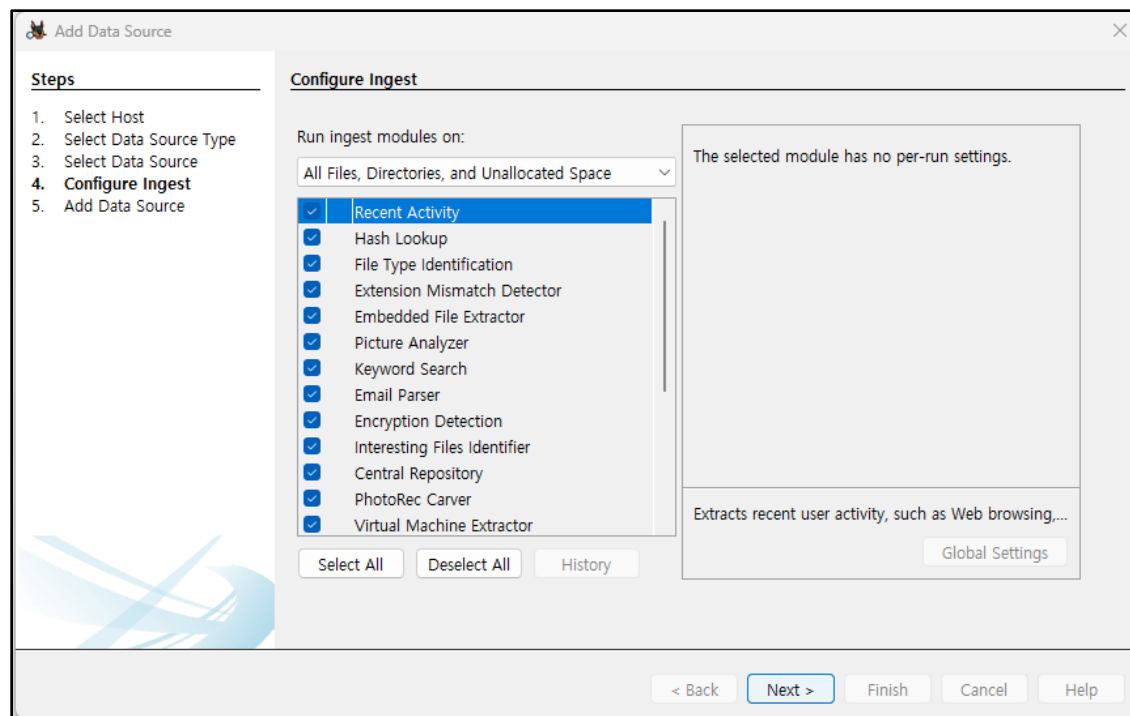
Evidence Tree	
20260209_DFRC51.E01	
Properties	
Evidence Source Path: C:\Users\Wgusdn\Desktop\20260209_DFRC51	
Evidence Type: Forensic Disk Image	
Disk	
Verification Hashes	
MD5 verification hash	7039515afaef479e0dff91a8b8aabc9
SHA1 verification hash	3b05693256fdb5da70fb4198df6d39e49784d9
Drive Geometry	
Bytes per Sector	512
Sector Count	468,862,128
Image	
Image Type	E01

[복제된 이미지(사본)]

[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - 개요

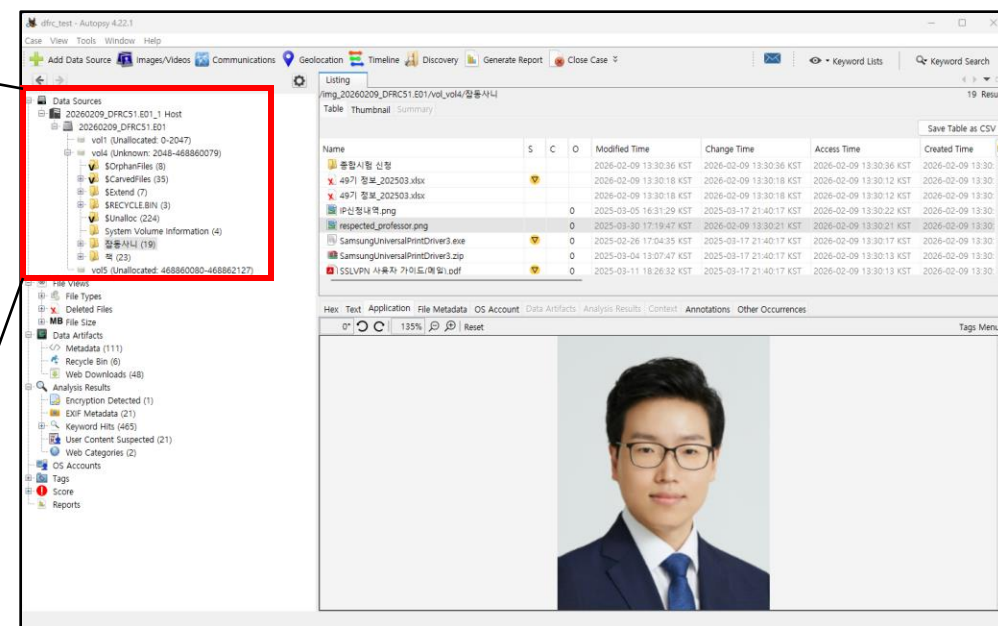
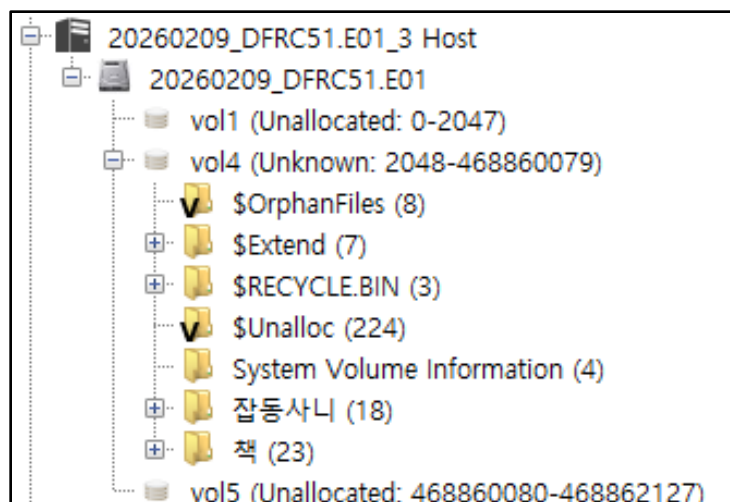
- Autopsy로 Case 생성 후 이미지 파일 연결
- Autopsy의 자동 분석 모듈인 'Ingest Module' 중 사용할 모듈을 체크 후 진행



[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - Data Sources

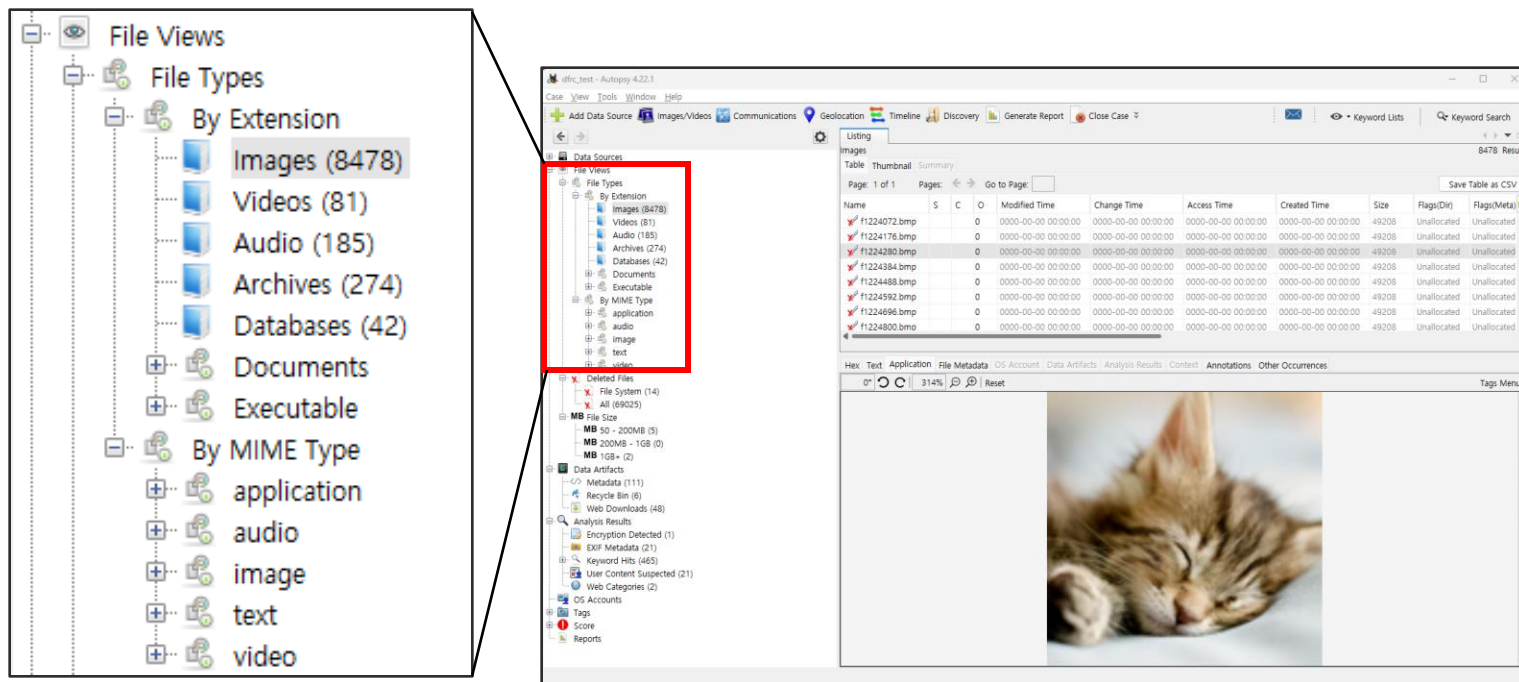
- 데이터를 트리 구조로 직접 보는 기능
- 물리 이미지: 인식된 파일 시스템으로 디스크 내 계층 구조를 그대로 탐색
- 논리 이미지: 계층 구조가 유지된 파일 단위로 탐색
- 탐색기 형태로 수동 분석 가능



[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - File Views

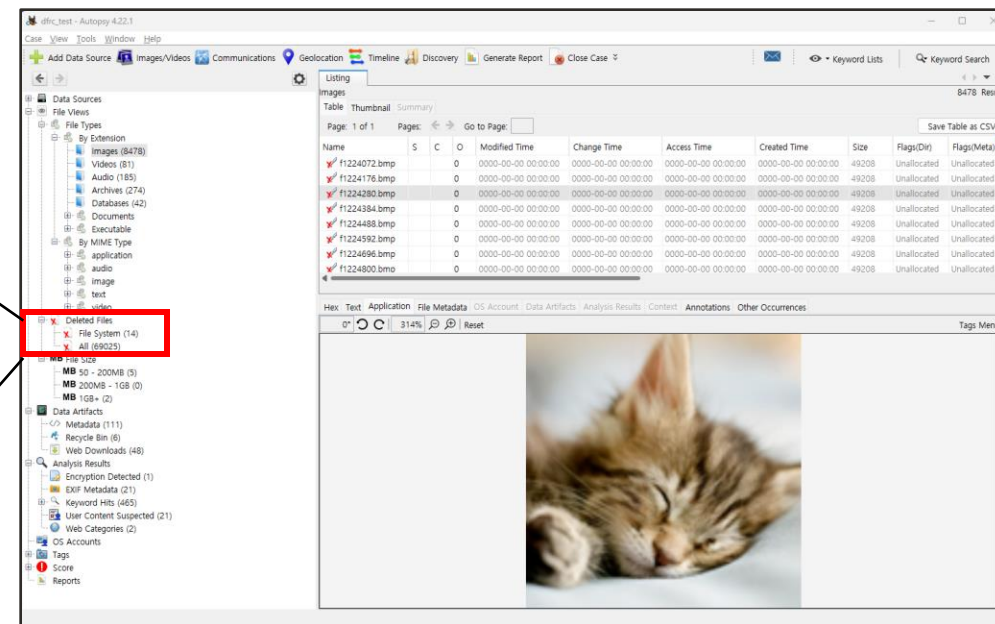
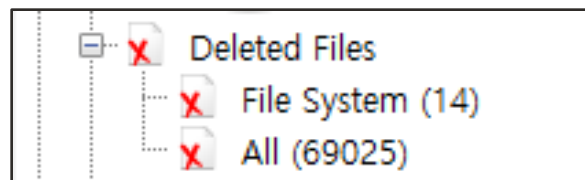
- 기본 제공되는 뷰어로, 타입 / 삭제된 파일 / 사이즈로 분류하여 파일을 볼 수 있는 기능
- File Types: 확장자 또는 MIME 타입을 기반으로 파일 타입 (이미지, 비디오 등)을 분류
 - Extension에 의한 분류
 - Images, Videos, Audio, Archives, Database
 - Documents
 - Executable
 - MIME Type에 의한 분류
 - application, audio, image, text, video



[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - File Views

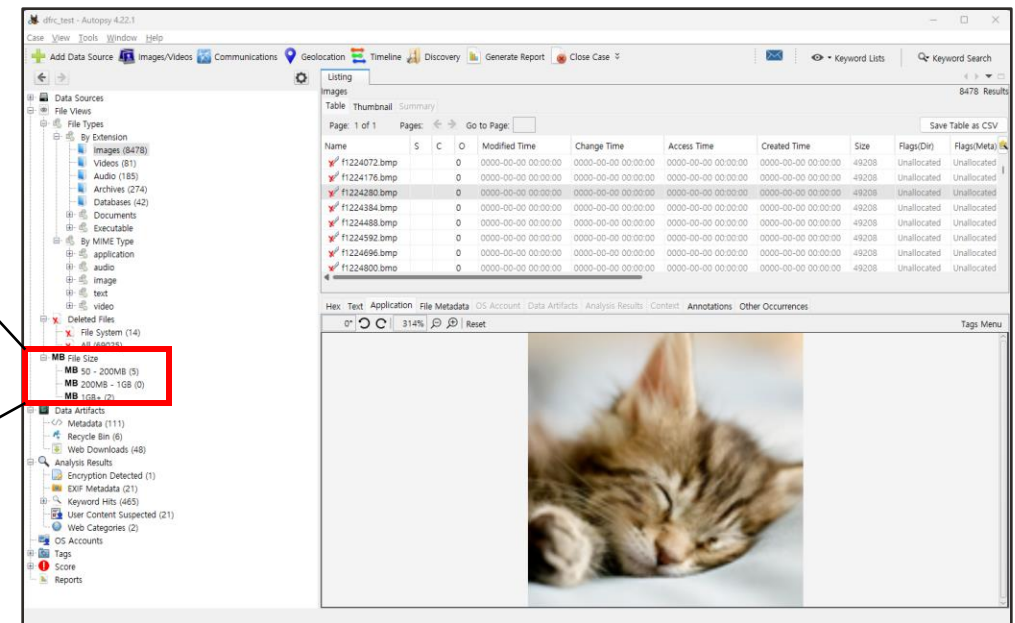
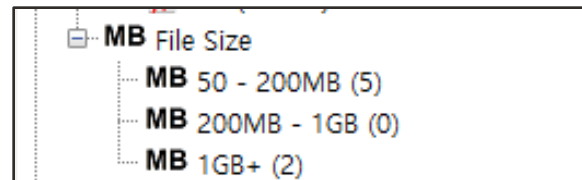
- 기본 제공되는 뷰어로, 타입 / 삭제된 파일 / 사이즈로 분류하여 파일을 볼 수 있는 기능
- Deleted Files: 삭제된 파일을 복구
 - File System: MFT Entry, Directory Entry를 순회하여 삭제된 파일을 식별하고 복구
 - All: 미할당영역을 스캔하여 시그니처 / 크기 기반으로 삭제된 파일을 식별하고 복구



[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - File Views

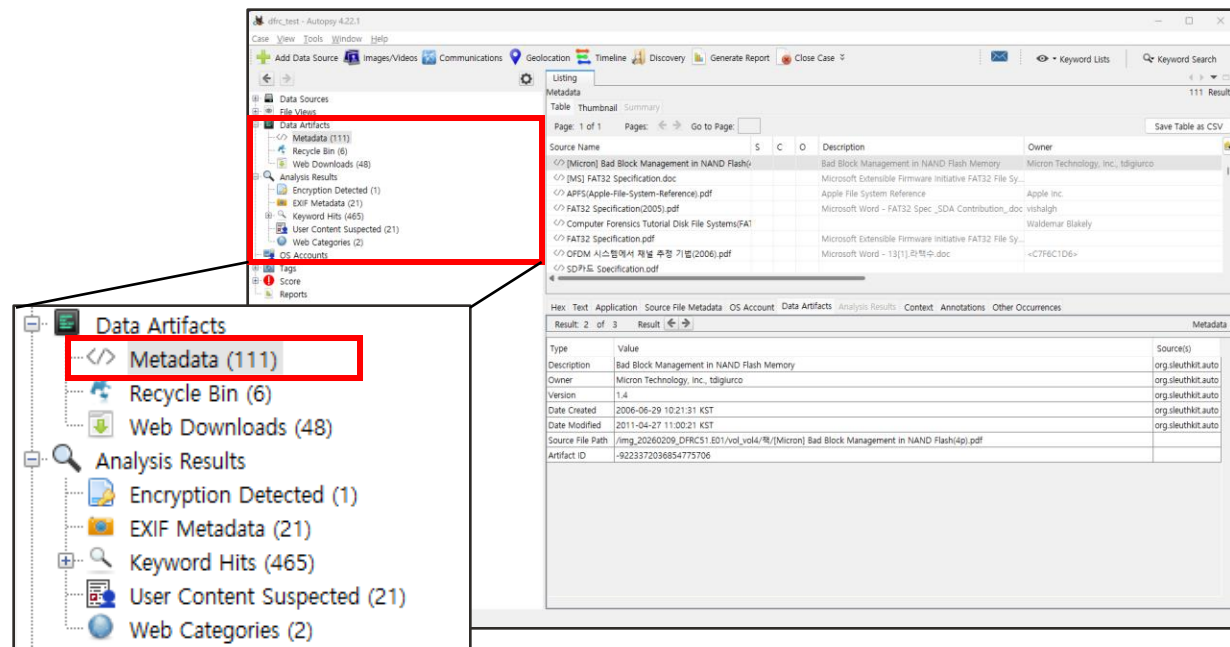
- 기본 제공되는 뷰어로, 타입 / 삭제된 파일 / 사이즈로 분류하여 파일을 볼 수 있는 기능
- File Size: 파일 크기로 분류
 - 50MB - 200MB / 200MB - 1GB / 1GB+
 - 파일 크기 분류는 'Discovery' 메뉴에서 설정 가능



[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - Data Artifacts / Analysis Results

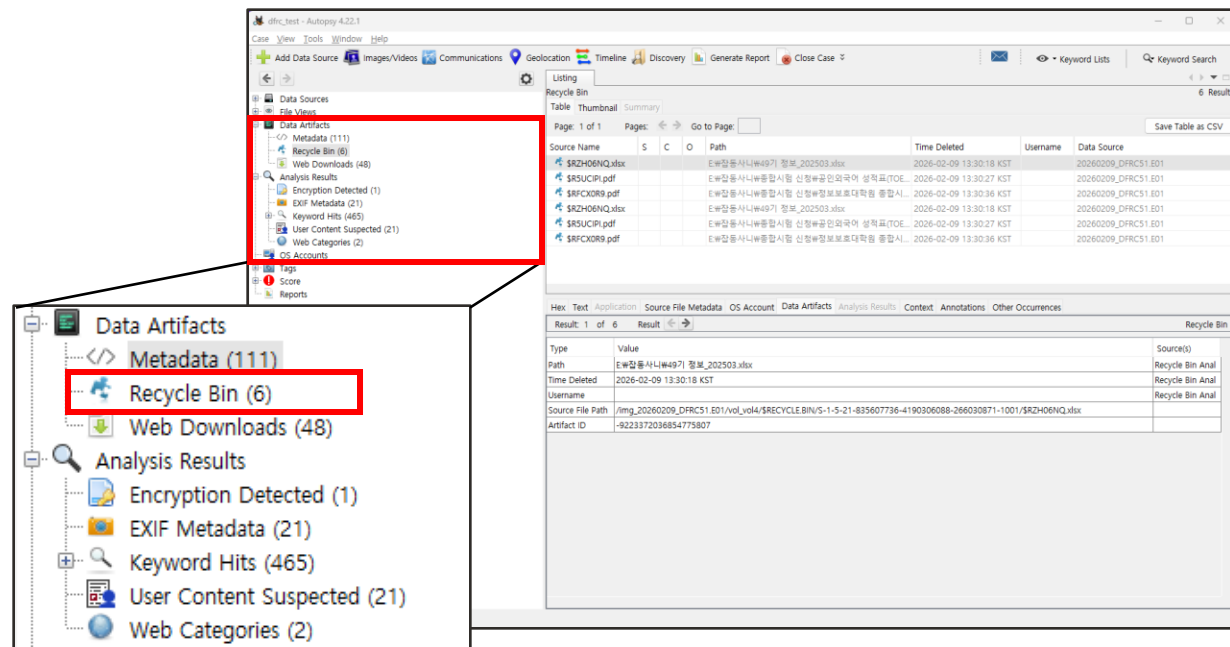
- 선택한 Ingest Module이 분석한 결과
- 파일의 메타 데이터, 웹 브라우저 활동 기록, 휴지통 정보 등이 포함
- Metadata: PDF, Office, 멀티미디어 등의 메타 데이터와 텍스트 내용을 파싱한 결과
 - 모듈: Keyword Search
 - 내부 텍스트를 탐지하고 추출하는 데에 Apache Tika 오픈소스 도구를 사용
 - 이 도구에서 파싱한 메타 데이터를 해당 항목에 분류



[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - Data Artifacts / Analysis Results

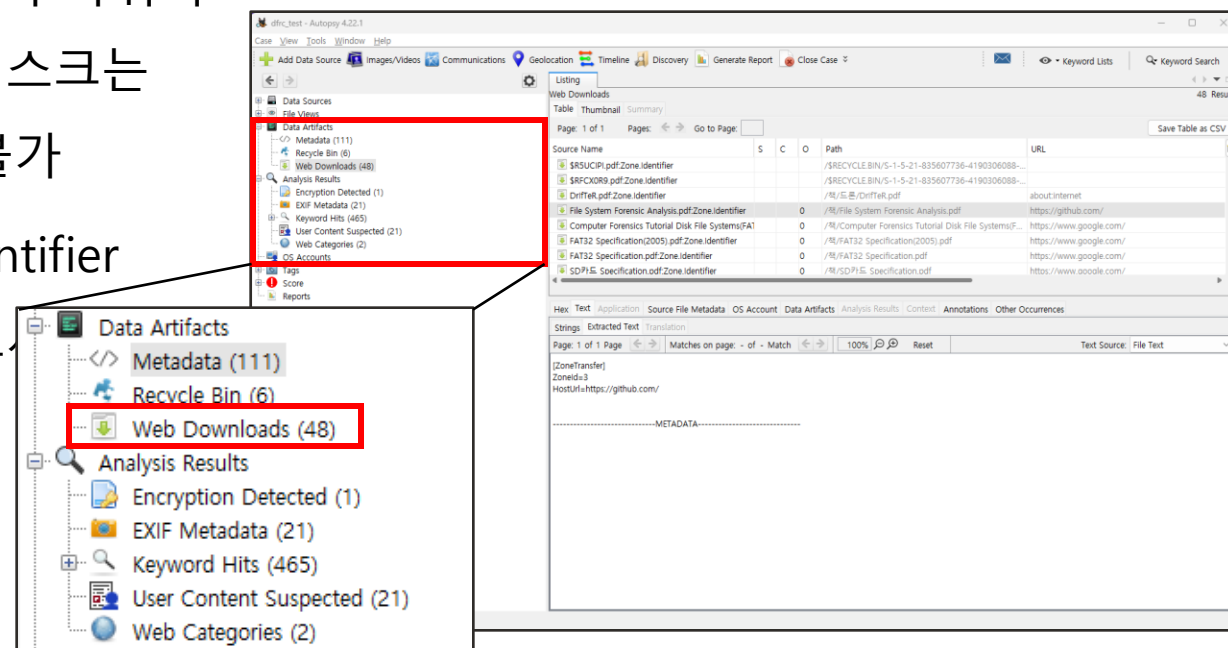
- Recycle Bin: 휴지통에 있는 파일들의 정보를 \$I에 저장된 메타 데이터와 결합한 결과
 - 모듈: Recent Activity
 - 사용자 활동 분석을 수행하는 오픈소스 도구 중 RegRipper의 동작 과정에서 휴지통 삭제 기록을 분석
 - 이때 \$I 정보를 결합하여 표시



[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - Data Artifacts / Analysis Results

- Web Downloads: 사용자가 웹 브라우저를 통해 다운로드 받은 파일을 분석한 결과
 - 모듈: Recent Activity
 - 내부적으로 웹 활동은 History DB, Cookie, LevelDB 등을 파싱 및 분석
 - 웹 브라우저는 일반적으로 사용자 홈 디렉터리 하위의 AppData에 아티팩트가 남으나, 분석 대상 디스크는 홈 디렉터리가 없어서 다른 결과들은 확인 불가
 - 다운로드 기록 또한 동일한 이유로 Zone.Identifier 외의 아티팩트가 없어서 Zone.Identifier만 표

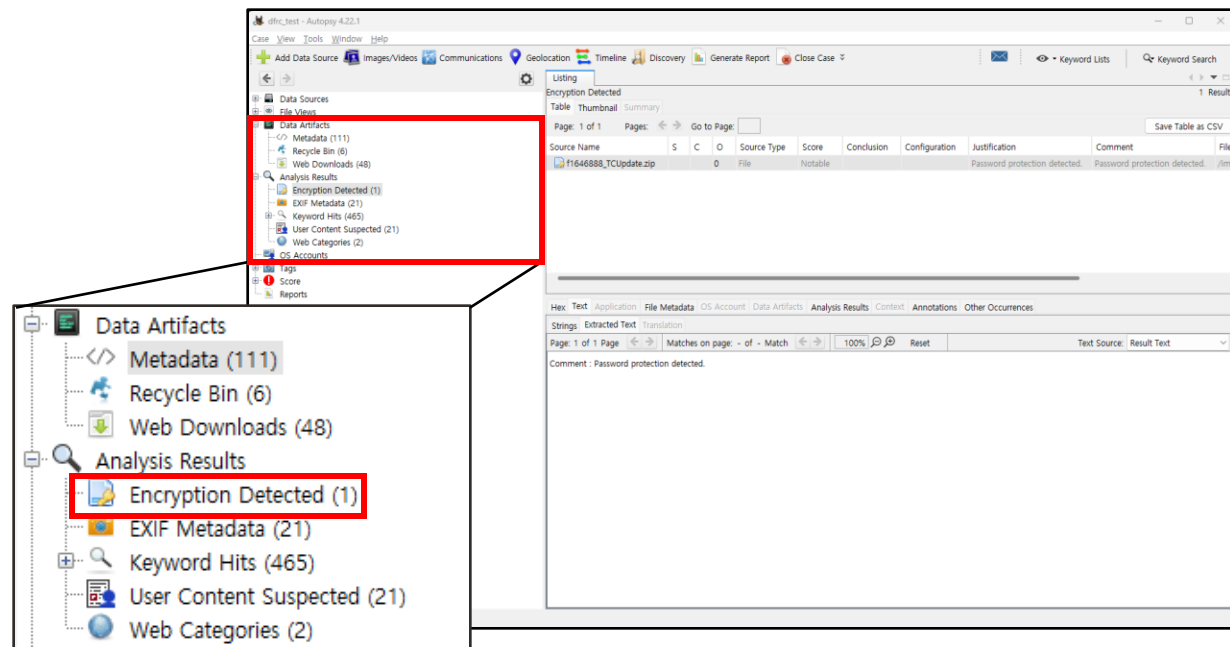


[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - Data Artifacts / Analysis Results

■ Encryption Detected: 비밀번호로 보호된 파일이나 볼륨을 식별한 결과

- 모듈: Encryption Detected, PhotoRec Carver
- Office / PDF / DB / Zip 등의 파일을 확인
- BitLocker로 암호화 된 볼륨도 확인 가능
- PhotoRec Carver 모듈로 미할당 영역에서 이미지, Zip 등의 파일을 식별하는 과정에서 암호화 된 파일이 식별된 경우에도 생성

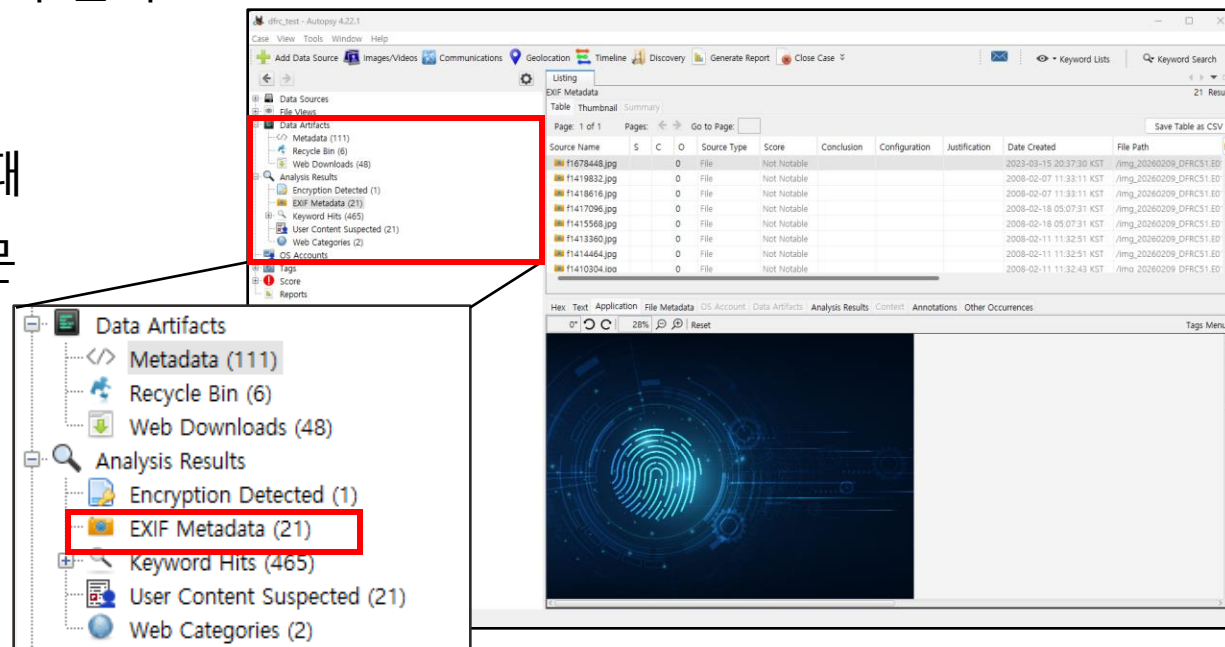


[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - Data Artifacts / Analysis Results

- EXIF Metadata: EXIF 메타데이터를 가지는 이미지 / 영상 등 멀티미디어의 메타데이터 분석 결과

- 모듈: Picture Analyzer
- PNG의 경우 EXIF Chunk를 가지지 않으면 분석 불가
- 일반적으로 JPEG 파일들이 출력
 - JPEG의 표준 포맷은 메타데이터를 담을 때 APP1 Marker (EXIF)를 기본으로 하기 때문

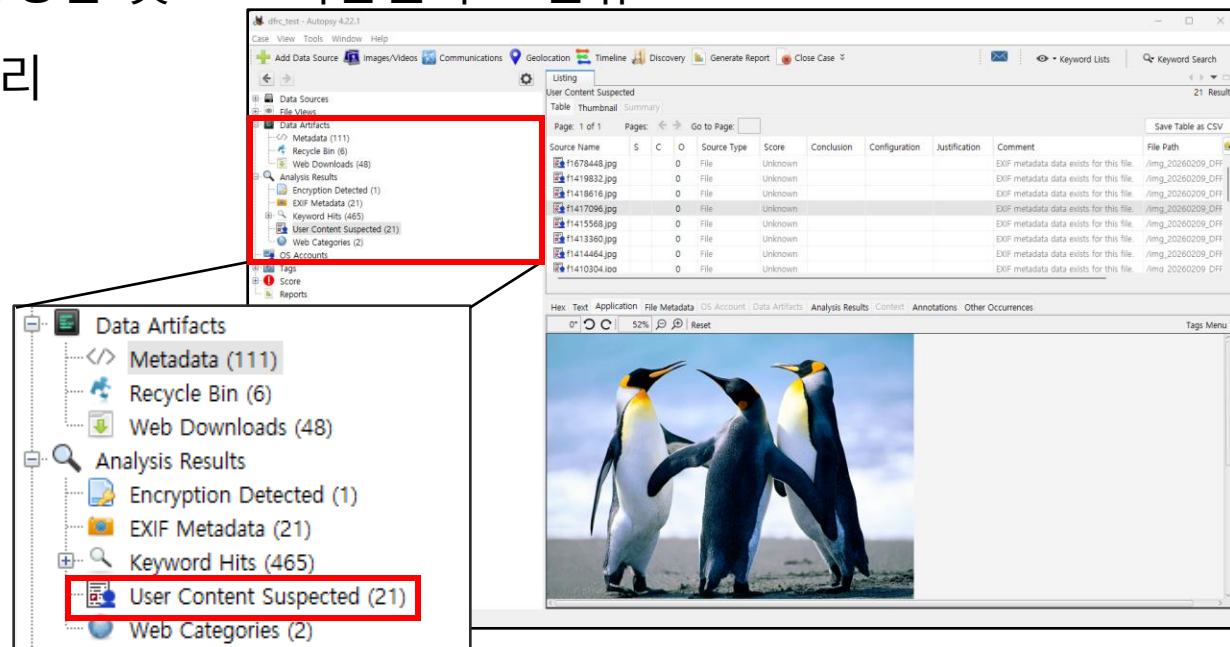


123

[실습] 디스크 덤프 및 분석[추가]

■ Autopsy - Data Artifacts / Analysis Results

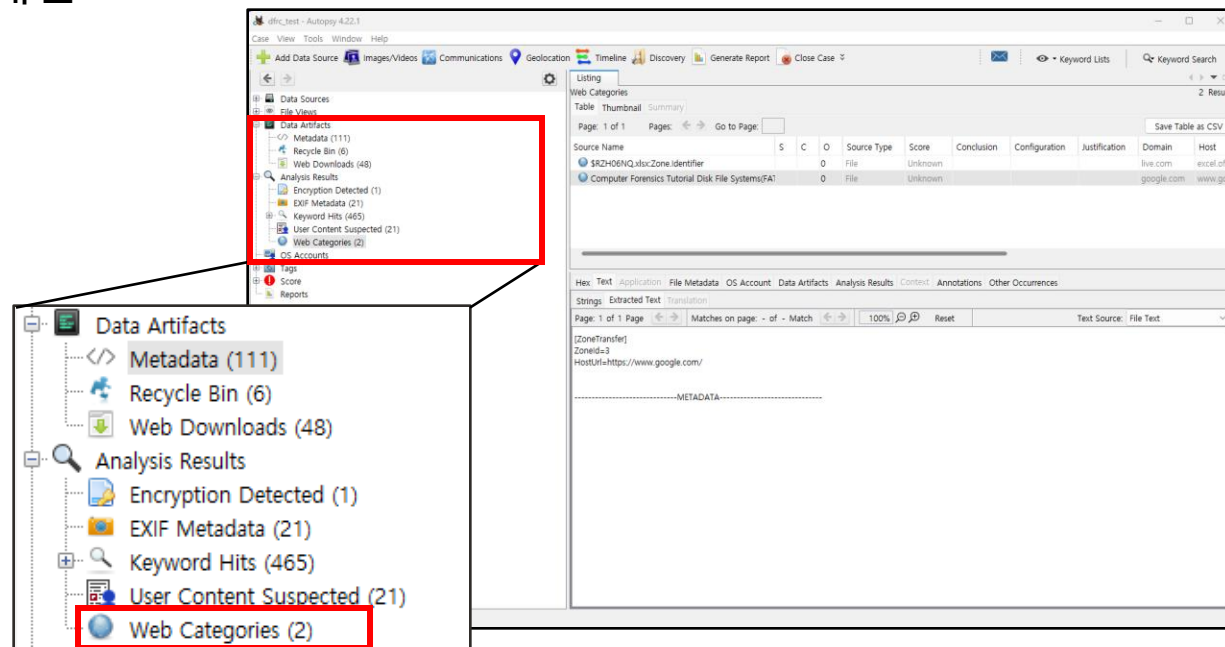
- User Content Suspected: 사용자가 생성한 것으로 의심되는 파일을 식별한 결과
 - 모듈: Picture Analyzer
 - 시스템이 아닌 사용자가 생성한 파일만 필터링한 결과
 - Autopsy에선 멀티미디어 파일을 사용자가 생성한 것으로 의심된다고 분류
 - Documents, Desktop 등의 사용자 홈 디렉터리 하위에 위치한 파일들도 함께 분류



[실습] 디스크 덤프 및 분석[추가]

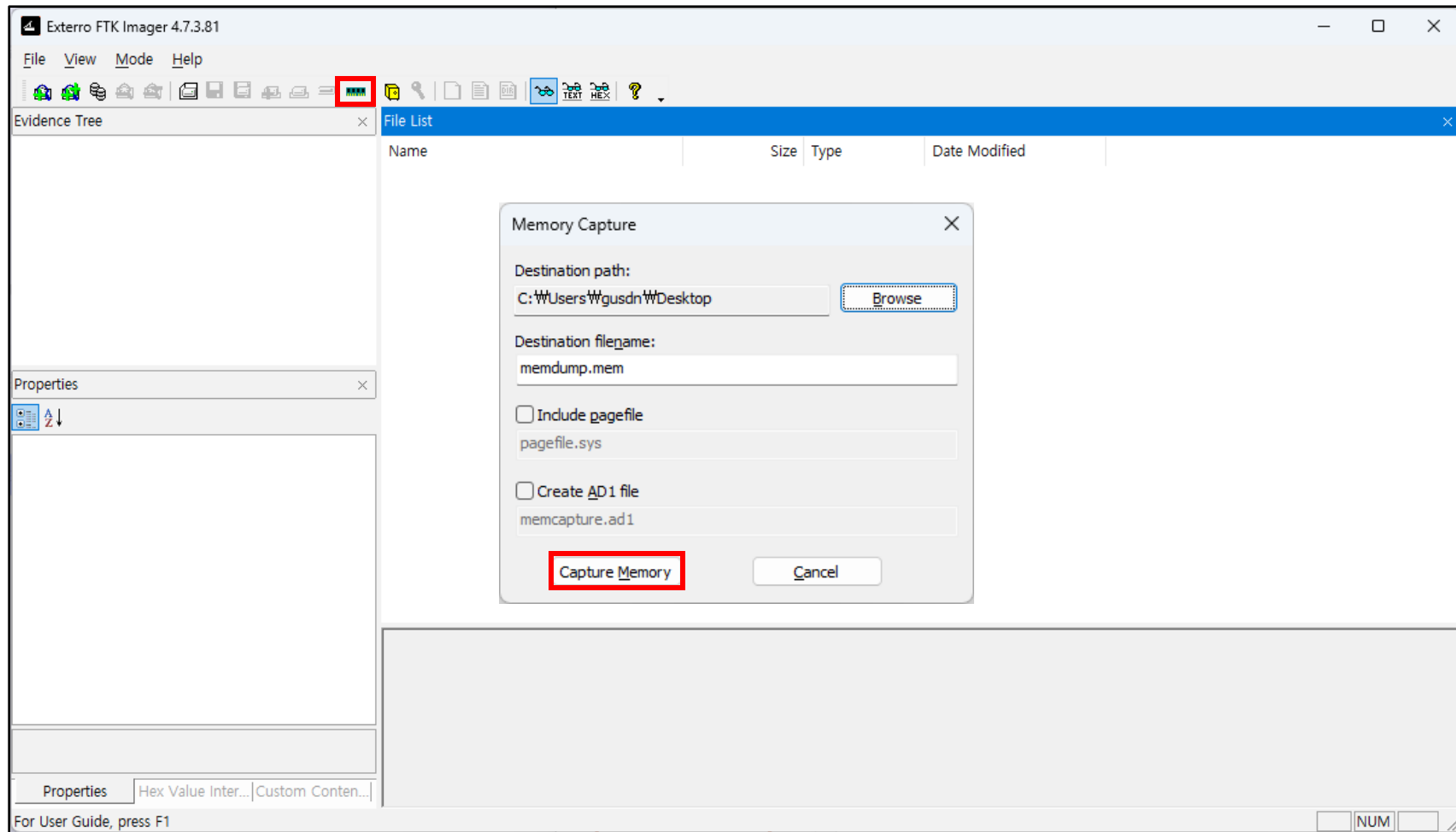
■ Autopsy - Data Artifacts / Analysis Results

- Web Categories: 분석가가 설정한 도메인 Suffix에 대한 웹 활동 분석 결과
 - 모듈: Recent Activity
 - Tools > Options > Custom Web Categories에서 도메인 Suffix 설정 가능
 - 설정을 하지 않더라도 built-in categories에 대한 결과가 표시 (google.com, live.com 등)



[실습] 메모리 덤프 및 분석

- FTK Imager를 사용하여 메모리 덤프



[실습] 메모리 덤프 및 분석[추가]

■ Volatility3를 사용한 메모리 분석 - 분석 대상 정보 확인

- 플러그인: windows.info
 - `python3 vol.py -f {memdump} windows.info`

```
Kernel Base      0xf801c5400000
DTB              0x1ae000
Symbols file:///C:/Users/cgumg/Desktop/%EB%8C%80%ED%95%99/.Minhyuk%20Tool/volatilit
y3/volatility3/symbols/windows/ntkrnlmp.pdb/A1C414A488BC6DE9308B5D3D7579D109-1.json
.xz
Is64Bit True
IsPAE False
layer_name       0 WindowsIntel32e
memory_layer     1 FileLayer
KdVersionBlock   0xf801c620a9a0
Major/Minor      15.26100
MachineType      34404
KeNumberProcessors 8
SystemTime       2026-02-20 08:56:01+00:00
NtSystemRoot     C:\Windows
NtProductType    NtProductWinNt
NtMajorVersion   10
NtMinorVersion   0
PE MajorOperatingSystemVersion 10
PE MinorOperatingSystemVersion 0
PE Machine       34404
PE TimeDateStamp Wed Aug 16 14:19:59 2062
```

항목	설명	해석
Is64Bit	현재 로드된 Symbol이 64bit 타입인지 여부	분석 대상 PC의 OS가 64bit 커널 (x64)
MachineType	현재 로드된 커널의 아키텍처 식별 값 / 34404 = x86-64 (AMD64)	
Major/Minor	Windows 빌드의 Major / Minor 버전 번호	빌드 버전 번호를 통해 Windows 11임을 확인
NTMajorVersion	Windows의 Major 버전	
SystemTime	당시의 시스템 시각	2026-02-20 17:56:01 KST
NtSystemRoot	Windows의 시스템 루트 경로	C:\Windows

[실습] 메모리 덤프 및 분석[추가]

■ Volatility3를 사용한 메모리 분석 - 프로세스 식별

■ 플러그인: windows.pslist

- python3 vol.py -f {memdump} windows.pslist
- EPROCESS의 이중 연결 리스트를 참조하여 프로세스 리스트를 순서대로 출력

6756	23468	cmd.exe	0xdf04faf7e080	1	-	6
19524	7808	Everything.exe	0xdf04e7d51080	18	-	-
28636	7808	notepad++.exe	0xdf04efee3080	3	-	-
30524	7808	Notepad.exe	0xdf04fd78d080	24	-	-
20596	24248	chrome.exe	0xdf04fc64a080	20	-	-
12760	3880	msedgewebview2	0xdf04fab16080	54	-	-
8644	12760	msedgewebview2	0xdf04ceb99080	7	-	-

항목	설명
PID	프로세스의 PID
PPID	부모 프로세스의 PID
ImageFileName	프로세스의 이름

[windows.pslist 주요 필드]

PID	PPID	ImageFileName
6756	23468	cmd.exe

[식별된 주요 프로세스]

[실습] 메모리 덤프 및 분석[추가]

■ Volatility3를 사용한 메모리 분석 - cmd.exe가 실행한 명령어 확인

■ 플러그인: windows.cmdscan

- python3 vol.py -f {memdump} windows.cmdscan
- conhost.exe를 대상으로 실행했던 명령어 히스토리 출력

```
** 26896 conhost.exe 0x28fc06db750 _COMMAND_HISTORY.CommandBucket_Comm
and_0 0x28fc06ddd10 DFRC51 Fighting
** 26896 conhost.exe 0x28fc06db750 _COMMAND_HISTORY.CommandBucket_Comm
and_1 0x28fc06ddd30 dir
** 26896 conhost.exe 0x28fc06db750 _COMMAND_HISTORY.CommandBucket_Comm
and_80 0x28fc06de710
** 26896 conhost.exe 0x28fc06db750 _COMMAND_HISTORY.CommandBucket_Comm
and_82 0x28fc06de750
```

항목	설명	해석
PID	프로세스의 PID	26896
Process	프로세스명	conhost.exe
Data	명령어 히스토리 출력	DFRC51 Fighting, dir 명령

[windows.cmdscan 주요 필드]

- conhost.exe: cmd.exe 등 콘솔 기반 프로그램의 화면 출력을 담당하는 콘솔 호스트 프로세스
 - 입력 이벤트 전달, 콘솔 버퍼 관리, 입·출력 기록

컴퓨터 분해·조립 및 기본 설정

[실습] PC 분해

■ 부품 기능 및 스펙 설명[추가]

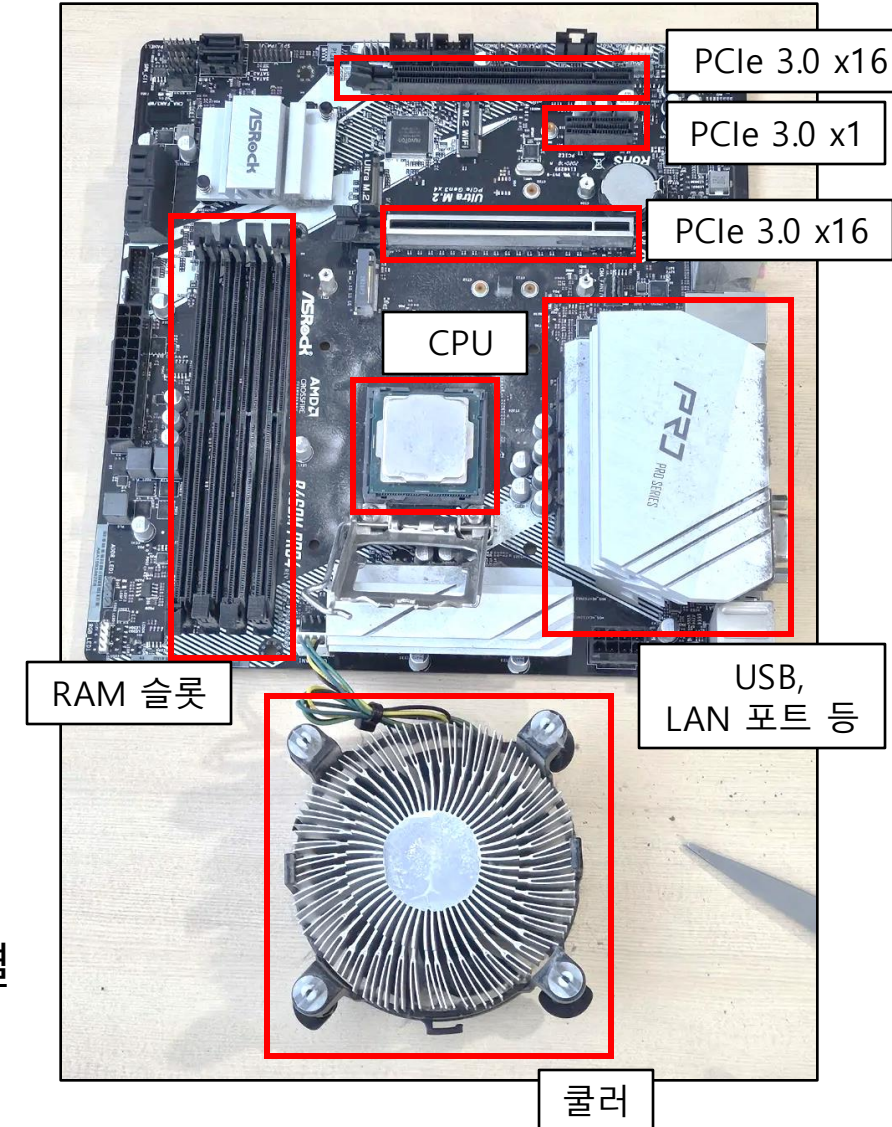
■ 메인보드: ASRock B460M Pro4

• 스펙

- CPU: 10th 세대 인텔 코어™ 프로세서
- RAM 슬롯: 4개, DDR4 지원
- 확장 슬롯: PCIe 3.0 x16 2개, PCIe 3.0 x1 1개
- 저장장치: SATA 포트 6개, M.2 폼팩터 2개
- USB 포트: USB 3.2 Gen1 7개

• 주요 기능

- CPU, 메모리, 그래픽 카드 등 모든 PC 부품을 물리적으로 연결
- 각 부품에 전원을 공급하고 데이터 흐름을 제어



[실습] PC 분해

■ 부품 기능 및 스펙 설명[추가]

- DDR: Samsung M378A2K43EB1-CWE (DDR4)
 - 스펙
 - 용량: 16 GB
 - 속도: 3200 Mbps
 - 전압: 1.2V
 - 주요 기능
 - 주기억장치로 CPU가 실행 중인 프로그램의 데이터 적재
 - 빠른 읽고 쓰기 기능으로 CPU의 작업 속도를 향상



[실습] PC 분해

- **부품 기능 및 스펙 설명****[추가]**
 - PSU: FSP HYPER K 600W
 - 스펙
 - 사이즈: 140 x 150 x 86mm
 - 출력: 600W
 - 입력: 200 ~ 240V
 - 주요 기능
 - 220V 가정용 전기를 PC용 전원으로 변환
 - 모든 부품에 안정적인 전력 공급



[실습] PC 분해

■ 부품 기능 및 스펙 설명[추가]

■ HDD: Western Digital WD40EZZAZ

• 스펙

- 용량: 4TB
- 인터페이스: SATA

• 주요 기능: 자기 디스크 기반 대용량 저장 장치

■ SSD: Samsung MZ-7KE256 850 Pro

• 스펙

- 용량: 256GB
- 인터페이스: SATA

• 주요 기능: NAND 플래시 메모리 기반 고속 저장 장치



[실습] PC 분해

■ 부품 기능 및 스펙 설명[추가]

■ GPU: INNO3D INNO3D GEFORCE GT 1030 2GB GDDR5

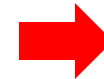
- 스펙
 - 최소 전력 요구 사항: 300W
 - 그래픽 메모리: 2GB
 - 버스: PCIe 3.0
 - 최대 디지털 해상도: 4096 x 2160
- 기능
 - 화면 출력과 고속 그래픽 연산 수행



[실습] PC 분해

■ 조립

- 파워 연결
- 메인보드 쿨러 장착 후 본체에 연결
- 메인보드 DDR 연결
- HDD, SSD, 그래픽 카드 연결 후 부팅



[실습] PC 조립 후 OS 설치 & IP 설정

■ 컴퓨터 부팅 및 OS(Windows) 설치[추가]

■ BIOS 부팅 과정

- 본 실습의 경우 저장장치에 OS가 이미 설치된 상태
- PC 재조립 후 전원 연결 → 기존 저장장치에 설치되어 있던 OS가 정상 부팅

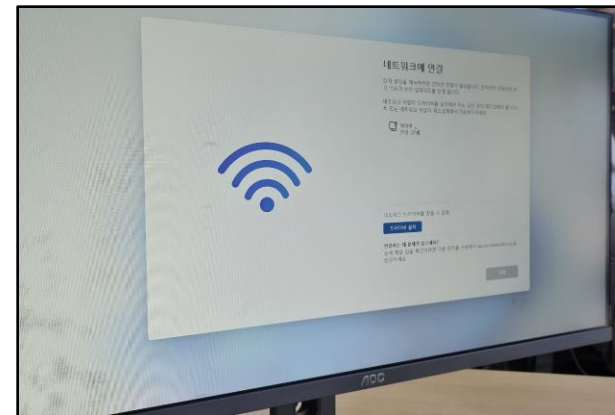
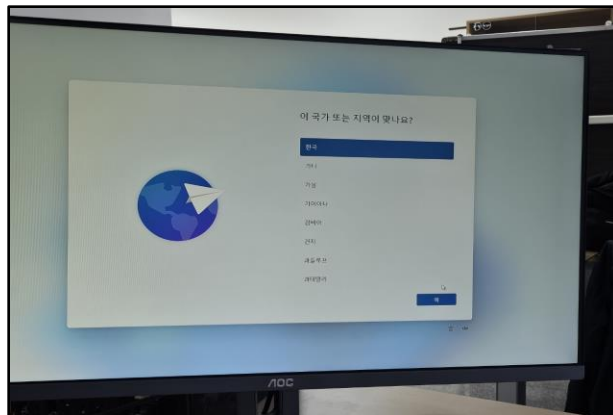
순서	주요 부품	세부 내용
1. 전원 공급	PSU, 메인보드	전력 연결 상태 확인 및 각 부품에 정격 전압 공급
2. POST 실행	메인보드, CPU, RAM, GPU 등	Power-On Self-Test 수행 (하드웨어 정상 여부 점검)
3. BIOS 초기화	메인보드 BIOS	부팅 순서 확인하고 저장장치 스캔해서 부트 로더 위치 확인
4. 부트 로더 실행	SSD/HDD, CPU, RAM	부트 섹터 및 Boot Manager RAM에 로드 → CPU가 부트 로더 코드 실행
5. OS 로딩	SSD/HDD, RAM	운영체제 커널 및 시스템 파일을 RAM에 로드
6. OS 실행	GPU, RAM	Windows 로고 및 로그인 화면 출력

[실습] PC 조립 후 OS 설치 & IP 설정

■ OS 및 IP 설정

■ 부팅 후 Windows 설정

- cmd창 → 'start ms-cxh:localhost' 커맨드 입력
 - ms-cxh: cxh를 호출하는 내부 URI 프로토콜, 계정 및 네트워크 설정 화면 실행
 - localhost: 온라인 MS 계정 흐름이 아닌, 로컬 전용 설정 흐름 실행
- MS 계정 연결 없이 로컬 계정 이용



[실습] PC 조립 후 OS 설치 & IP 설정

■ OS 설치 및 IP 설정

■ IP 설정

- '설정 → 네트워크 및 인터넷 → 이더넷'에서 IP 설정 편집
 - 수동으로 IPv4 설정
 - IP 주소: 163.152.127.197
 - 서브넷 마스크: 255.255.255.0
 - 게이트웨이: 163.152.127.1
 - 기본 설정 DNS: 163.152.1.1

IP 설정 편집

수동

IPv4

☒ 켜

IP 주소

163.152.127.197

서브넷 마스크

255.255.255.0

게이트웨이

163.152.127.1

기본 설정 DNS

163.152.1.1

HTTPS를 통한 DNS

끔

[실습] PC 조립 후 OS 설치 & IP 설정

■ OS 설치 및 IP 설정

■ IP 설정

- cmd창에서 'ipconfig' 명령어로 설정 확인
 - IPv4 주소: 163.152.127.197
 - 서브넷 마스크: 255.255.255.0
 - 게이트웨이: 163.152.127.1



```
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DFRC>ipconfig

Windows IP 구성

이더넷 어댑터 이더넷:

    연결별 DNS 접미사 . . . . . :
    링크-로컬 IPv6 주소 . . . . . : fe80::96ea:eb4c:f67b:534%7
    IPv4 주소 . . . . . : 163.152.127.197
    서브넷 마스크 . . . . . : 255.255.255.0
    기본 게이트웨이 . . . . . : 163.152.127.1
```

THANK YOU
for Listening!



Digital Forensic Research Center

고 려 대 학 교 정 보 보 호 연 구 원
디 지 털 포 렌 식 연 구 센 터

dfrc.korea.ac.kr

Questions?

